bayesian-statistics: the Bayesian Statistics program — user guide

bayesian-statistics-ai999x 1.1.0, Rust build: command line, desktop app and browser. **Creator / engineer:** Ibnul Jaif Farabi · **Company:** AI999X · support@ai999x.com

Bayesian Statistics covers Bayes' theorem, conjugate priors and posteriors, credible intervals, Bayes factors, Monte Carlo estimation, uncertainty propagation, sensitivity analysis and decision analysis. This guide is about `bayesian-statistics`, the model as a self-contained program for Windows and Linux: 78 capabilities in 12 topics, answered on your own computer with no network, no Python and nothing else to install. The same compiled model runs in the AI999X desktop app and in the browser on the model's page (https://www.ai999x.com/models/bayesian-statistics-ai999x/); the Python package (`pip install bayesian-statistics-ai999x`) has its own guide. Every command output printed here is what the program printed.

Contents

- 1. Get it: desktop app, command line, browser, Python
- 2. Quick start
- 3. Writing inputs
- 4. Outcomes and exit codes
- 5. Scripts and agents: batch and JSON
- 6. Figures
- 7. In a web page or Node.js: WebAssembly
- 8. Every capability

1. Get it

Version 1.4.0 of the AI999X desktop app and of the command-line bundle. Checksums for every file: https://downloads.ai999x.com/desktop/1.4.0/SHA256SUMS; the newest version is always under https://downloads.ai999x.com/desktop/latest/. Full instructions: https://www.ai999x.com/desktop-cli/

what	system	download
Desktop app: Bayesian Statistics	Windows x86-64	https://downloads.ai999x.com/desktop/1.4.0/ai999x-bayesian-statistics-1.4.0-windows-x86_64.zip
Desktop app: Bayesian Statistics	Linux x86-64	https://downloads.ai999x.com/desktop/1.4.0/ai999x-bayesian-statistics-1.4.0-linux-x86_64.tar.gz
Desktop app: all models	Windows x86-64	https://downloads.ai999x.com/desktop/1.4.0/ai999x-1.4.0-windows-x86_64.zip
Desktop app: all models	Linux x86-64	https://downloads.ai999x.com/desktop/1.4.0/ai999x-1.4.0-linux-x86_64.tar.gz
Command line	Windows x86-64	https://downloads.ai999x.com/desktop/1.4.0/ai999x-cli-1.4.0-windows-x86_64.zip
Command line	Linux x86-64	https://downloads.ai999x.com/desktop/1.4.0/ai999x-cli-1.4.0-linux-x86_64.tar.gz

Desktop app

Bayesian Statistics in its own app: search, questions in words, input forms, live figures and explanations. Windows: Download ai999x-bayesian-statistics-1.4.0-windows-x86_64.zip, right-click it → Extract All, then open the folder and start ai999x-bayesian-statistics.exe. If Windows SmartScreen asks, choose More info → Run anyway. Linux:

```
curl -LO https://downloads.ai999x.com/desktop/1.4.0/ai999x-bayesian-statistics-1.4.0-linux-
x86_64.tar.gz
curl -LO https://downloads.ai999x.com/desktop/1.4.0/SHA256SUMS
sha256sum -c --ignore-missing SHA256SUMS
tar xzf ai999x-bayesian-statistics-1.4.0-linux-x86_64.tar.gz
./ai999x-bayesian-statistics-1.4.0-linux-x86_64/ai999x-bayesian-statistics
```

It opens on this model: type a question or pick a capability, fill in the form and press Compute. The AI999X desktop app with every model works the same way (choose the model first). Windows: Download ai999x-1.4.0-windows-x86_64.zip, right-click it → Extract All, then open the folder and start ai999x.exe. If Windows SmartScreen asks, choose More info → Run anyway. Linux:

```
curl -LO https://downloads.ai999x.com/desktop/1.4.0/ai999x-1.4.0-linux-x86_64.tar.gz
curl -LO https://downloads.ai999x.com/desktop/1.4.0/SHA256SUMS
sha256sum -c --ignore-missing SHA256SUMS
tar xzf ai999x-1.4.0-linux-x86_64.tar.gz
./ai999x-1.4.0-linux-x86_64/ai999x
```

Command line — Linux

The bundle holds one program per model; `bayesian-statistics` is this one. The checksum line should print OK.

```
curl -LO https://downloads.ai999x.com/desktop/1.4.0/ai999x-cli-1.4.0-linux-x86_64.tar.gz
curl -LO https://downloads.ai999x.com/desktop/1.4.0/SHA256SUMS
sha256sum -c --ignore-missing SHA256SUMS
mkdir -p ~/.local/share && tar xzf ai999x-cli-1.4.0-linux-x86_64.tar.gz -C ~/.local/share
echo 'export PATH="$HOME/.local/share/ai999x-cli-1.4.0-linux-x86_64:$PATH"' >> ~/.bashrc && source
~/.bashrc
bayesian-statistics --version
```

Command line — Windows (PowerShell)

The fourth line should print True.

```
$F = "ai999x-cli-1.4.0-windows-x86_64.zip"
Invoke-WebRequest "https://downloads.ai999x.com/desktop/1.4.0/$F" -OutFile $F
Invoke-WebRequest "https://downloads.ai999x.com/desktop/1.4.0/SHA256SUMS" -OutFile SHA256SUMS
(Get-FileHash $F -Algorithm SHA256).Hash.ToLower() -eq ((Select-String -SimpleMatch $F
SHA256SUMS).Line -split ' ')[0]
Expand-Archive $F -DestinationPath "$env:LOCALAPPDATA\AI999X" -Force
$D = "$env:LOCALAPPDATA\AI999X\ai999x-cli-1.4.0-windows-x86_64"; $env:Path += ";$D"
[Environment]::SetEnvironmentVariable("Path", [Environment]::GetEnvironmentVariable("Path", "User")
+ ";$D", "User")
bayesian-statistics --version
```

In your browser

Open https://www.ai999x.com/models/bayesian-statistics-ai999x/, choose a capability and press Compute in your browser: the same program runs on your device as WebAssembly; nothing is sent anywhere.

Python

```
pip install bayesian-statistics-ai999x
```

2. Quick start

2.1 Check the install

Prints the model, its version and that this is the Rust build.

```
$ bayesian-statistics --version
bayesian-statistics-ai999x 1.0.3 (Rust build, ax-engine 0.1.0)
```

2.2 List the capabilities

Every capability the program answers: its id (what you pass to solve) and its name.

```
$ bayesian-statistics list
ab_test_probability_beta                Bayesian A/B test: probability that A beats B
bayes_action_minimum_expected_loss      Bayes action: minimum posterior expected loss
bayes_factor_binomial_simple            Bayes factor for two simple binomial hypotheses
bayes_factor_normal_mean                Bayes factor for a normal mean (known variance)
bayes_factor_point_null_binomial        Bayes factor for a point null on a proportion (beta prior
under H1)
bayes_theorem                           Bayes' theorem for an event and its complement
beta_binomial_credible_interval         Credible interval for a proportion (beta-binomial)
beta_binomial_evidence                  Marginal likelihood (evidence) of binomial data under a
beta prior
… 69 more
```

2.3 What a capability needs

The equation or method, every input and output with its SI unit. Read this before solve: it tells you the input names.

```
$ bayesian-statistics info diagnostic_test_predictive_values
Positive and negative predictive values of a test
  Bayes' theorem with the total probability of a positive result
  in   prevalence prevalence (pre-test probability of the condition) [1]
  in   sensitivity sensitivity P(positive | condition) [1]
  in   specificity specificity P(negative | no condition) [1]
  out  ppv      positive predictive value P(condition | positive) [1]
  out  npv      negative predictive value P(no condition | negative) [1]
  out  p_positive probability of a positive result [1]
  out  false_discovery_rate 1 - PPV [1]
  out  false_omission_rate 1 - NPV [1]
```

2.4 Solve

Inputs as name=value, each with any unit of the right kind. The answer lists every output, the method used, each input converted to SI, and the conditions the answer assumes.

```
$ bayesian-statistics solve diagnostic_test_predictive_values prevalence=0.01 sensitivity=0.9
specificity=0.95
ppv = 0.1538461538
npv = 0.9989378651
p_positive = 0.0585
false_discovery_rate = 0.8461538462
false_omission_rate = 0.001062134891
  by: Positive and negative predictive values of a test  (Bayes' theorem with the total probability
of a positive result)
  prevalence = 0.01   []
  sensitivity = 0.9   []
  specificity = 0.95   []
  assumes: probabilities between 0 and 1; both results possible
```

2.5 Machine-readable answer

--json prints the same answer as one line of JSON (values in SI): for scripts and other programs.

```
$ bayesian-statistics solve diagnostic_test_predictive_values prevalence=0.01 sensitivity=0.9
specificity=0.95 --json
{"outcome":"SOLVED","message":"","value":0.153846153846,"unit":"","solved_for":"ppv","si":
[["prevalence",0.01],["sensitivity",0.9],["specificity",0.95]],"values":[["ppv",0.153846153846,""],
["npv",0.998937865109,""],["p_positive",0.0585,""],["false_discovery_rate",0.846153846154,""],
["false_omission_rate",0.00106213489113,""]],"target":"ppv","phrase":"positive predictive value
P(condition | positive)","assumptions":["probabilities between 0 and 1","both results
possible"],"name":"Positive and negative predictive values of a test","equation":"Bayes' theorem
with the total probability of a positive result"}
```

2.6 When an input is missing

No number is guessed: the answer names what is missing, shows a complete call, and the program exits with status 1.

```
$ bayesian-statistics solve diagnostic_test_predictive_values prevalence=0.01 sensitivity=0.9
NEEDS_INPUT: diagnostic_test_predictive_values needs specificity (specificity P(negative | no
condition)). Example: solve('diagnostic_test_predictive_values', prevalence=0.01, sensitivity=0.9,
specificity=0.95).
```

2.7 Many questions at once

One JSON request per line in, one JSON answer per line out, in order: for scripts, notebooks and agents (the program starts once). In PowerShell: Get-Content cases.jsonl | bayesian-statistics batch

`cases.jsonl`:

```
{"key": "diagnostic_test_predictive_values", "inputs": {"prevalence": 0.01, "sensitivity": 0.9,
"specificity": 0.95}}
{"key": "beta_binomial_posterior", "inputs": {"alpha": 2.0, "beta": 2.0, "successes": "7",
"trials": "10"}}
```

```
$ bayesian-statistics batch < cases.jsonl
{"outcome":"SOLVED","message":"","value":0.153846153846,"unit":"","solved_for":"ppv","si":
[["prevalence",0.01],["sensitivity",0.9],["specificity",0.95]],"values":[["ppv",0.153846153846,""],
["npv",0.998937865109,""],["p_positive",0.0585,""],["false_discovery_rate",0.846153846154,""],
["false_omission_rate",0.00106213489113,""]],"target":"ppv","phrase":"positive predictive value
P(condition | positive)","assumptions":["probabilities between 0 and 1","both results
possible"],"name":"Positive and negative predictive values of a test","equation":"Bayes' theorem
with the total probability of a positive result"}
{"outcome":"SOLVED","message":"","value":0.642857142857,"unit":"","solved_for":"posterior_mean","si":
[["alpha",2.0],["beta",2.0]],"values":[["post_alpha",9.0,""],["post_beta",5.0,""],
["posterior_mean",0.642857142857,""],["posterior_variance",0.015306122449,""],
["posterior_sd",0.123717914826,""]],"target":"posterior_mean","phrase":"posterior alpha = alpha +
k","assumptions":["alpha, beta > 0","0 <= k <= n"],"name":"Beta-binomial conjugate
posterior","equation":"conjugate update of the beta prior with the binomial likelihood"}
```

3. Writing inputs

Inputs are written `name=value`, one per argument; `bayesian-statistics info <capability>` lists the names. Quote an argument that holds a space or brackets (bash, zsh and PowerShell all accept double quotes). The answer always states each input as it was read, converted to SI, so a misread unit is visible at once.

kind	example	how it is read
a number with a unit	`"R=0.128 nm"`	any unit of the same kind; converted to SI
a number in SI	`R=1.28e-10`	no unit means the SI unit shown by info
a choice	`structure=FCC`	one of the values info lists
a list or vector	`"v=[1, 2, 3]" or "v=1, 2, 3"`	brackets keep it a list
a matrix	`"A=[[1, 2], [3, 4]]"`	one list per row
true / false	`inclusive=true`	for yes/no options

An equation can be solved for any of its quantities: give the others and add `--target <symbol>`. Without `--target`, the program solves for the quantity the capability is named after.

4. Outcomes and exit codes

The program never guesses a number: when it cannot answer, it says why and exits with status 1, so a script can tell an answer from a refusal without reading the text.

outcome	meaning	exit status
SOLVED	the answer: every output with its unit, the method, the inputs in SI, the conditions assumed	0
NEEDS_INPUT	an input is missing; the message names it and shows a complete call	1
INVALID_INPUT	a value or unit of the wrong kind, an unknown input name, or a value outside the conditions	1
AMBIGUOUS	the inputs allow two different answers; say which one is meant	1

outcome	meaning	exit status
OUT_OF_SCOPE	the model has no capability for this, or the result would not be meaningful	1
(usage)	a command the program does not know: it prints its usage	2

5. Scripts and agents: batch and JSON

`bayesian-statistics solve ... --json` prints one answer as one line of JSON. `bayesian-statistics batch` reads one request per line on standard input and writes one JSON answer per line, in order, starting the program once for any number of requests. A request is `{"key": "<capability>", "inputs": {"name": "value", ...}, "target": null}`. In PowerShell, pipe the file in: `Get-Content cases.jsonl | bayesian-statistics batch`. `bayesian-statistics catalog` prints every capability with its inputs as JSON, to build forms or tool definitions.

answer field	meaning
outcome	one of the outcomes above
message	why there is no answer (empty when SOLVED)
value, unit	the main result in SI units
values	every output: [symbol, value, unit] (procedures)
target	the quantity solved for
si	each numeric input converted to SI: [symbol, value]
assumptions	the conditions the answer relies on
name, equation	the capability and its equation or method

```
import json, subprocess

reqs = [{"key": "<capability>", "inputs": {"name": "value"}}]
out = subprocess.run(["bayesian-statistics", "batch"], input="\n".join(map(json.dumps, reqs)),
                     capture_output=True, text=True, check=True).stdout
answers = [json.loads(line) for line in out.splitlines()]
```

6. Figures

`bayesian-statistics figures <capability> name=value ... --out DIR` draws the live figures for those inputs and writes each as an SVG file into DIR (an existing folder); without `--out` the SVG text comes back as JSON. The desktop app and the model page draw the same figures as soon as you press Compute.

7. In a web page or Node.js: WebAssembly

The same model is a WebAssembly module, `bayesian_statistics_ai999x.wasm` (https://www.ai999x.com/compute/rust/bayesian_statistics_ai999x.wasm), with a small loader, `ax-wasm.js` (https://www.ai999x.com/compute/rust/ax-wasm.js). It runs in a browser, a Web Worker or Node.js, with no server.

```
<script src="ax-wasm.js"></script>
<script>
  AXWasm.load("bayesian_statistics_ai999x.wasm").then(function (m) {
    var a = m.solve({key: "<capability>", inputs: {name: "value"}});
    console.log(a.outcome, a.values || a.value);
    console.log(m.catalog().capabilities.length + " capabilities");
  });
</script>
```

8. Every capability

By topic. For each: what it does, what it needs and gives, when it applies, and a command with what the program printed for it. `bayesian-statistics info <id>` shows the same inputs and outputs in the terminal.

Bayes factors and model comparison (6)

Bayes factor for a normal mean (known variance) — `bayes_factor_normal_mean`

BF10 for H1: mu ~ N(mu0, tau^2) against H0: mu = mu0, from the sample mean of n observations with known sigma; BF01 and the posterior probability of H1 at equal prior odds.

$$BF_{01} = \frac{p(\mu_0 \mid \bar{x}, H_1)}{p(\mu_0 \mid H_1)}$$

symbol		meaning	SI unit
`sample_mean`	in	sample mean	—
`n`	in	sample size (>= 1)	—
`sigma`	in	known standard deviation (> 0)	—
`mu0`	in	null value mu0	—
`prior_sd`	in	prior standard deviation tau of mu under H1 (> 0)	—
`bf10`	out	Bayes factor H1 : H0	—
`bf01`	out	1/BF10	—
`log10_bf10`	out	log10 BF10	—
`posterior_h1`	out	P(H1 | data) with prior odds 1	—

Applies when: sigma, tau > 0; n >= 1.

```
$ bayesian-statistics solve bayes_factor_normal_mean sample_mean=0 n=1 sigma=1 mu0=0 prior_sd=1
bf10 = 0.7071067812
bf01 = 1.414213562
log10_bf10 = -0.1505149978
posterior_h1 = 0.4142135624
  by: Bayes factor for a normal mean (known variance)  (Savage-Dickey density ratio of the
posterior and prior at mu0)
  sample_mean = 0   []
  sigma = 1   []
  mu0 = 0   []
  prior_sd = 1   []
  assumes: sigma, tau > 0; n >= 1
```

What the worked example means: This calculation determines the strength of evidence for a normal mean differing from a specified value using a known variance. By comparing the posterior and prior densities at the null value, it yields a Bayes factor of 0.707107 for the alternative model against the null. The corresponding inverse Bayes factor is 1.41421, and the logarithmic scale value is -0.150515. With equal prior odds, the posterior probability of the alternative model is estimated as 0.414214. These metrics rely on a sample size of 1, a sample mean of 0, and a prior standard deviation of 1, assuming a known population standard deviation of 1.

Bayes factor for a point null on a proportion (beta prior under H1) — `bayes_factor_point_null_binomial`

BF10 for H1: p ~ Beta(alpha, beta) against H0: p = p0 after k successes in n trials (default uniform prior), with BF01 and the posterior probability of H1 at equal prior odds.

$$BF_{10} = \frac{B(\alpha + k,\ \beta + n - k) / B(\alpha, \beta)}{p_0^k (1 - p_0)^{n-k}}$$

symbol		meaning	SI unit
`successes`	in	successes k	—
`trials`	in	trials n (>= 1)	—
`p0`	in	null value p0	—
`alpha`	in	Beta alpha under H1 (default 1) (optional)	—
`beta`	in	Beta beta under H1 (default 1) (optional)	—
`bf10`	out	Bayes factor H1 : H0	—
`bf01`	out	1/BF10	—
`log10_bf10`	out	log10 BF10	—
`posterior_h1`	out	P(H1 | data) with prior odds 1	—

Applies when: 0 < p0 < 1; alpha, beta > 0; 0 <= k <= n.

```
$ bayesian-statistics solve bayes_factor_point_null_binomial p0=0.5 successes=2 trials=2 alpha=1
beta=1
bf10 = 1.333333333
bf01 = 0.75
log10_bf10 = 0.1249387366
posterior_h1 = 0.5714285714
  by: Bayes factor for a point null on a proportion (beta prior under H1)  (ratio of the beta-
binomial marginal likelihood to the binomial likelihood at p0)
  p0 = 0.5   []
  alpha = 1   []
  beta = 1   []
  assumes: 0 < p0 < 1; alpha, beta > 0; 0 <= k <= n
```

What the worked example means: This calculation determines the strength of evidence for a beta-distributed proportion against a fixed null value. Given 2 successes in 2 trials, with a null hypothesis of 0.5 and prior parameters alpha of 1 and beta of 1, the resulting Bayes factor is 1.33333. The reciprocal, 0.75, represents evidence for the null. Under equal prior odds, the posterior probability that the alternative model is true is 0.571429.

Bayes factor for two simple binomial hypotheses — `bayes_factor_binomial_simple`

Bayes factor BF10 for H1: p = p1 against H0: p = p0 after k successes in n trials, its log10 and the posterior probability of H1 with equal prior odds.

$$BF_{10} = \left(\frac{p_1}{p_0}\right)^{k}\left(\frac{1-p_1}{1-p_0}\right)^{n-k}$$

symbol		meaning	SI unit
`successes`	in	successes k	—
`trials`	in	trials n (>= 1)	—
`p0`	in	success probability under H0	—
`p1`	in	success probability under H1	—
`bf10`	out	Bayes factor H1 : H0	—
`log10_bf10`	out	log10 of BF10	—
`posterior_h1`	out	P(H1 | data) with prior odds 1	—

Applies when: 0 < p0, p1 < 1; 0 <= k <= n.

```
$ bayesian-statistics solve bayes_factor_binomial_simple p0=0.5 p1=0.7 successes=14 trials=20
bf10 = 5.184417905
log10_bf10 = 0.7147000018
posterior_h1 = 0.8383032946
  by: Bayes factor for two simple binomial hypotheses  (likelihood ratio of the two simple
hypotheses)
  p0 = 0.5   []
  p1 = 0.7   []
  assumes: 0 < p0, p1 < 1; 0 <= k <= n
```

What the worked example means: This calculation compares two simple hypotheses about a success probability using observed data consisting of 14 successes in 20 trials. The likelihood ratio yields a Bayes factor of 5.18442 for the hypothesis with probability 0.7 against the hypothesis with probability 0.5. This evidence translates to a log base ten scale value of 0.7147. Assuming equal prior odds between the two hypotheses, the resulting posterior probability that the first hypothesis is true given the data is 0.838303.

Marginal likelihood (evidence) of binomial data under a beta prior — `beta_binomial_evidence`

Prior predictive probability of k successes in n trials when p ~ Beta(alpha, beta), and its logarithm.

$$p(k) = \binom{n}{k} \frac{B(\alpha + k,\ \beta + n - k)}{B(\alpha, \beta)}$$

symbol		meaning	SI unit
`alpha`	in	prior Beta alpha (> 0)	—
`beta`	in	prior Beta beta (> 0)	—
`successes`	in	number of successes k	—
`trials`	in	number of trials n	—
`marginal_likelihood`	out	P(k | n, alpha, beta)	—
`log_marginal_likelihood`	out	its natural log	—

Applies when: alpha, beta > 0; 0 <= k <= n.

```
$ bayesian-statistics solve beta_binomial_evidence alpha=1 beta=1 successes=7 trials=10
marginal_likelihood = 0.09090909091
log_marginal_likelihood = -2.397895273
  by: Marginal likelihood (evidence) of binomial data under a beta prior  (closed form with log-beta functions)
  alpha = 1   []
  beta = 1   []
  assumes: alpha, beta > 0; 0 <= k <= n
```

What the worked example means: This calculation determines the probability of observing exactly 7 successes out of 10 independent Bernoulli trials, assuming the underlying success rate follows a Beta distribution with parameters alpha equal to 1 and beta equal to 1. The resulting value, 0.0909091, represents the evidence for the data given the prior model. Its natural logarithm is computed as -2.3979, which is often more numerically stable for large numbers of trials.

Posterior model probabilities from log evidences — `posterior_model_probabilities`

Posterior probabilities of several models from their log marginal likelihoods and prior model probabilities (default equal), computed stably in log space.

$$P(M_i \mid D) = \frac{P(D \mid M_i) P(M_i)}{\sum_j P(D \mid M_j) P(M_j)}$$

symbol		meaning	SI unit
`log_evidences`	in	natural-log marginal likelihoods of the models	—
`priors`	in	prior model probabilities (default equal) (optional)	—
`posterior`	out	posterior model probabilities	—
`best_model`	out	number of the most probable model	—
`best_probability`	out	its posterior probability	—

Applies when: prior probabilities sum to 1.

```
$ bayesian-statistics solve posterior_model_probabilities "log_evidences=[-10, -10, -10]"
posterior = [0.333333333333,0.333333333333,0.333333333333]
best_model = 1
best_probability = 0.3333333333
  by: Posterior model probabilities from log evidences  (log-sum-exp normalization of log prior +
log evidence)
  assumes: prior probabilities sum to 1
```

What the worked example means: This calculation determines the posterior probability for each competing model by combining their log evidences with prior probabilities. The result shows that model 1 has the highest likelihood given the data, with a probability of 0.333333, while the other models have lower corresponding probabilities derived from the normalized exponential terms.

Posterior probability of a hypothesis from a Bayes factor — `posterior_from_bayes_factor`

Posterior probability of H1 from the Bayes factor BF10 and the prior probability of H1 (default 1/2).

$$\frac{P(H_1 \mid D)}{P(H_0 \mid D)} = BF_{10}\,\frac{P(H_1)}{P(H_0)}$$

symbol		meaning	SI unit
`bayes_factor`	in	Bayes factor BF10 in favour of H1 (> 0)	—
`prior`	in	prior probability of H1 (default 0.5) (optional)	—
`posterior`	out	posterior probability of H1	—
`posterior_odds`	out	posterior odds H1 : H0	—
`posterior_h0`	out	posterior probability of H0	—

Applies when: BF10 > 0; 0 < prior < 1.

```
$ bayesian-statistics solve posterior_from_bayes_factor bayes_factor=6 prior=0.5
posterior = 0.8571428571
posterior_odds = 6
posterior_h0 = 0.1428571429
  by: Posterior probability of a hypothesis from a Bayes factor  (posterior odds = BF10 x prior
odds)
  bayes_factor = 6   []
  prior = 0.5   []
  assumes: BF10 > 0; 0 < prior < 1
```

What the worked example means: This calculation determines the updated probability of hypothesis H1 after observing data. It multiplies the provided Bayes factor, which quantifies the evidence strength for H1 over H0, by the prior odds to yield the posterior odds. These odds are then converted into a probability, resulting in a posterior value of 0.857143. This process translates abstract model comparison metrics into actionable probabilistic statements about which hypothesis is more likely given the evidence.

Bayes' theorem and conditional probability (5)

Bayes' theorem for an event and its complement — `bayes_theorem`

The posterior probability P(A|B) from the prior P(A) and the likelihoods P(B|A) and P(B|not A).

$$P(A \mid B) = \frac{P(B \mid A)\,P(A)}{P(B \mid A)\,P(A) + P(B \mid \neg A)\,(1 - P(A))}$$

symbol		meaning	SI unit
`p_a`	in	prior probability P(A)	—
`p_b_given_a`	in	likelihood P(B|A)	—
`p_b_given_not_a`	in	likelihood P(B|not A)	—
`p_a_given_b`	out	posterior probability P(A|B)	—
`p_b`	out	total probability P(B)	—
`p_not_a_given_b`	out	P(not A|B) = 1 - P(A|B)	—

Applies when: probabilities between 0 and 1; P(B) > 0.

```
$ bayesian-statistics solve bayes_theorem p_a=0.3 p_b_given_a=0.8 p_b_given_not_a=0.1
p_a_given_b = 0.7741935484
p_b = 0.31
p_not_a_given_b = 0.2258064516
  by: Bayes' theorem for an event and its complement  (law of total probability for P(B), then
Bayes' theorem)
  p_a = 0.3   []
  p_b_given_a = 0.8   []
  p_b_given_not_a = 0.1   []
  assumes: probabilities between 0 and 1; P(B) > 0
```

What the worked example means: This calculation determines the updated probability of event A given evidence B. It uses the law of total probability to find the marginal likelihood of B, which equals 0.31, combining the prior probability 0.3 with the conditional probabilities 0.8 and 0.1. The resulting posterior probability for A is 0.774194, while the complementary probability for not A is 0.225806. This process rationalizes belief revision by weighing new evidence against existing assumptions.

Law of total probability — `total_probability`

P(B) = sum of P(B|A_i) P(A_i) over a partition A_1, ..., A_k (and the joint probabilities).

$$P(B) = \sum_i P(B \mid A_i)\,P(A_i)$$

symbol		meaning	SI unit
`priors`	in	probabilities P(A_i) of the partition (sum to 1)	—
`conditionals`	in	conditional probabilities P(B|A_i), one per part	—
`p_b`	out	total probability P(B)	—
`joint`	out	joint probabilities P(B and A_i)	—

Applies when: the P(A_i) sum to 1; same number of conditionals as parts.

```
$ bayesian-statistics solve total_probability "priors=[0.5, 0.3, 0.2]" "conditionals=[0.01, 0.02,
0.05]"
p_b = 0.021
joint = [0.005,0.006,0.01]
  by: Law of total probability  (weighted sum over the partition)
  assumes: the P(A_i) sum to 1; same number of conditionals as parts
```

What the worked example means: This calculation determines the overall likelihood of an outcome by combining conditional probabilities with their respective prior probabilities across a partition. The result, 0.021, represents the marginal probability of the target event, while [0.005, 0.006, 0.01] details the specific joint contributions for each scenario within the partition.

Positive and negative predictive values of a test — `diagnostic_test_predictive_values`

The probability of the condition after a positive result (PPV) and of no condition after a negative one (NPV), from prevalence, sensitivity and specificity.

$$\mathrm{PPV} = \frac{\mathrm{Se}\,\pi}{\mathrm{Se}\,\pi + (1-\mathrm{Sp})(1-\pi)}, \quad \mathrm{NPV} = \frac{\mathrm{Sp}\,(1-\pi)}{\mathrm{Sp}\,(1-\pi) + (1-\mathrm{Se})\,\pi}$$

symbol		meaning	SI unit
`prevalence`	in	prevalence (pre-test probability of the condition)	—
`sensitivity`	in	sensitivity P(positive | condition)	—
`specificity`	in	specificity P(negative | no condition)	—
`ppv`	out	positive predictive value P(condition | positive)	—
`npv`	out	negative predictive value P(no condition | negative)	—
`p_positive`	out	probability of a positive result	—
`false_discovery_rate`	out	1 - PPV	—
`false_omission_rate`	out	1 - NPV	—

Applies when: probabilities between 0 and 1; both results possible.

```
$ bayesian-statistics solve diagnostic_test_predictive_values prevalence=0.01 sensitivity=0.9
specificity=0.95
ppv = 0.1538461538
npv = 0.9989378651
p_positive = 0.0585
false_discovery_rate = 0.8461538462
false_omission_rate = 0.001062134891
  by: Positive and negative predictive values of a test  (Bayes' theorem with the total probability
of a positive result)
  prevalence = 0.01   []
  sensitivity = 0.9   []
  specificity = 0.95   []
  assumes: probabilities between 0 and 1; both results possible
```

What the worked example means: This calculation determines the probability that a person actually has the condition given a positive test result, known as the positive predictive value, which is 0.153846. It also finds the probability that a person does not have the condition given a negative result, the negative predictive value, at 0.998938. These values are derived from the pre-test probability of the condition, or prevalence 0.01, along with the test's ability to correctly identify those with the disease (sensitivity 0.9) and those without it (specificity 0.95). The overall chance of getting a positive result on this test is 0.0585.

Posterior after repeated independent tests — `sequential_test_posterior`

Probability of the condition after several conditionally independent test results (k positive and m negative), by multiplying the pre-test odds by LR+ for each positive and LR- for each negative.

$$\text{odds}_{post} = \frac{\pi}{1-\pi}\,(LR^{+})^{k}\,(LR^{-})^{m}$$

symbol		meaning	SI unit
`prevalence`	in	prevalence (pre-test probability of the condition)	—
`sensitivity`	in	sensitivity P(positive | condition)	—
`specificity`	in	specificity P(negative | no condition)	—
`positives`	in	number of positive results	—
`negatives`	in	number of negative results (default 0) (optional)	—
`posterior`	out	probability of the condition after all results	—
`posterior_odds`	out	its odds	—

Applies when: 0 < prevalence < 1; results conditionally independent given the status.

```
$ bayesian-statistics solve sequential_test_posterior prevalence=0.01 sensitivity=0.9
specificity=0.95 positives=2 negatives=0
posterior = 0.7659574468
posterior_odds = 3.272727273
  by: Posterior after repeated independent tests  (odds form, applied once per result (in log
space))
  prevalence = 0.01   []
  sensitivity = 0.9   []
  specificity = 0.95   []
  assumes: 0 < prevalence < 1; results conditionally independent given the status
```

What the worked example means: This calculation determines the updated probability of a condition after processing multiple independent test results. It begins with the pre-test probability, represented here as 0.01, and converts it into odds. For each positive result, the odds are multiplied by the likelihood ratio for positivity, and for each negative result, they are multiplied by the likelihood ratio for negativity. The final computed posterior odds are 3.27273, which translate to a posterior probability of 0.765957. This method allows clinicians to integrate cumulative evidence from repeated screenings or diagnostic tests into a single coherent risk estimate.

Posterior over several discrete hypotheses — `discrete_bayes_posterior`

Posterior probabilities of k hypotheses from their priors and the likelihood of the observed data under each; the evidence and the most probable hypothesis (MAP).

$$P(H_i \mid D) = \frac{P(D \mid H_i)\,P(H_i)}{\sum_j P(D \mid H_j)\,P(H_j)}$$

symbol		meaning	SI unit
`priors`	in	prior probabilities of the hypotheses (sum to 1)	—
`likelihoods`	in	likelihood of the data under each hypothesis	—
`posterior`	out	posterior probabilities	—
`evidence`	out	marginal probability of the data	—
`map_hypothesis`	out	number (1-based) of the most probable hypothesis	—
`map_probability`	out	its posterior probability	—

Applies when: priors sum to 1; likelihoods >= 0, not all zero where the prior is positive.

```
$ bayesian-statistics solve discrete_bayes_posterior "priors=[0.5, 0.3, 0.2]" "likelihoods=[0.1,
0.4, 0.7]"
posterior = [0.161290322581,0.387096774194,0.451612903226]
evidence = 0.31
map_hypothesis = 3
map_probability = 0.4516129032
  by: Posterior over several discrete hypotheses  (Bayes' theorem: multiply prior by likelihood and
normalize)
  assumes: priors sum to 1; likelihoods >= 0, not all zero where the prior is positive
```

What the worked example means: This calculation applies Bayes theorem to update prior beliefs using observed data likelihoods. The resulting posterior probabilities indicate how confidence shifts from the initial priors to the final assessment, with the highest probability assigned to the most probable hypothesis at 0.451613. The total evidence value of 0.31 represents the marginal probability of observing the given data across all hypotheses.

Conjugate priors (17)

Bayesian A/B test: probability that A beats B — `ab_test_probability_beta`

P(p_A > p_B) for independent Beta posteriors of two success probabilities (alpha_b a whole number, as after a uniform or any integer-parameter prior).

$$P(p_B > p_A) = \sum_{i=0}^{\alpha_B - 1} \frac{B(\alpha_A + i,\ \beta_A + \beta_B)}{(\beta_B + i)\, B(1 + i,\ \beta_B)\, B(\alpha_A, \beta_A)}$$

symbol		meaning	SI unit
`alpha_a`	in	posterior alpha of A	—
`beta_a`	in	posterior beta of A	—
`alpha_b`	in	posterior alpha of B (whole number)	—
`beta_b`	in	posterior beta of B	—
`prob_a_greater`	out	P(p_A > p_B)	—
`prob_b_greater`	out	P(p_B > p_A)	—

Applies when: all parameters > 0; alpha_b a whole number.

```
$ bayesian-statistics solve ab_test_probability_beta alpha_a=1 beta_a=1 alpha_b=1 beta_b=1
prob_a_greater = 0.5
prob_b_greater = 0.5
  by: Bayesian A/B test: probability that A beats B  (exact finite sum of beta functions (Miller's
formula))
  alpha_a = 1   []
  beta_a = 1   []
  alpha_b = 1   []
  beta_b = 1   []
  assumes: all parameters > 0; alpha_b a whole number
```

What the worked example means: This calculation determines the exact Bayesian probability that the success rate of variant A exceeds that of variant B, given their independent Beta posterior distributions. The result shows a probability of 0.5 for A being better and 0.5 for B being better. These values are derived from Miller's formula using the posterior parameters alpha and beta for both groups, providing a direct measure of superiority without relying on frequentist p-values.

Beta-binomial conjugate posterior — `beta_binomial_posterior`

Update a Beta(alpha, beta) prior on a success probability with k successes in n trials: posterior parameters, mean, variance and standard deviation.

$$p \mid k \sim \mathrm{Beta}(\alpha + k,\ \beta + n - k), \quad E[p \mid k] = \frac{\alpha + k}{\alpha + \beta + n}$$

symbol		meaning	SI unit
`alpha`	in	prior Beta alpha (> 0)	—
`beta`	in	prior Beta beta (> 0)	—
`successes`	in	number of successes k	—
`trials`	in	number of trials n	—
`post_alpha`	out	posterior alpha = alpha + k	—
`post_beta`	out	posterior beta = beta + n - k	—

symbol		meaning	SI unit
`posterior_mean`	out	posterior mean	—
`posterior_variance`	out	posterior variance	—
`posterior_sd`	out	posterior standard deviation	—

Applies when: alpha, beta > 0; 0 <= k <= n.

```
$ bayesian-statistics solve beta_binomial_posterior alpha=2 beta=2 successes=7 trials=10
post_alpha = 9
post_beta = 5
posterior_mean = 0.6428571429
posterior_variance = 0.01530612245
posterior_sd = 0.1237179148
  by: Beta-binomial conjugate posterior  (conjugate update of the beta prior with the binomial
likelihood)
  alpha = 2   []
  beta = 2   []
  assumes: alpha, beta > 0; 0 <= k <= n
```

What the worked example means: This calculation updates a Beta distribution prior using binomial likelihood data. The posterior parameters are determined by adding the number of successes to the prior alpha, resulting in 9, and adding the number of failures to the prior beta, resulting in 5. The expected value of the success probability given this updated belief is 0.642857, while the spread of the uncertainty is quantified by a standard deviation of 0.123718.

Dirichlet-multinomial conjugate posterior — `dirichlet_multinomial_posterior`

Update a Dirichlet(alpha_1, ..., alpha_k) prior on category probabilities with category counts: posterior concentrations, means and variances.

$$\theta \mid n \sim \mathrm{Dir}(\alpha_1 + n_1, \ldots, \alpha_k + n_k), \quad E[\theta_j] = \frac{\alpha_j + n_j}{\alpha_0 + n}$$

symbol		meaning	SI unit
`alpha`	in	prior concentrations alpha_j (> 0)	—
`counts`	in	observed category counts	—
`post_alpha`	out	alpha_j + n_j	—
`posterior_means`	out	posterior means (alpha_j + n_j)/(alpha_0 + n)	—
`posterior_variances`	out	posterior variances	—

Applies when: alpha_j > 0; counts whole numbers >= 0; same length.

```
$ bayesian-statistics solve dirichlet_multinomial_posterior "alpha=[1, 1, 1]" "counts=[10, 5, 3]"
post_alpha = [11.0,6.0,4.0]
posterior_means = [0.52380952381,0.285714285714,0.190476190476]
posterior_variances = [0.0113378684807,0.00927643784787,0.00700886415172]
  by: Dirichlet-multinomial conjugate posterior  (conjugate update of the Dirichlet prior with the
multinomial likelihood)
  assumes: alpha_j > 0; counts whole numbers >= 0; same length
```

What the worked example means: This calculation updates the belief about category probabilities by combining the prior Dirichlet concentrations with the observed multinomial counts. The resulting posterior means, such as [0.52381, 0.285714, 0.190476], represent the expected probability for each category after accounting for new data. This shift reflects how much weight the evidence carries relative to the initial prior assumptions defined by [1, 1, 1].

Dirichlet-multinomial probability of a count vector (evidence) — `dirichlet_multinomial_evidence`

Prior predictive probability (marginal likelihood) of observed category counts when the category probabilities have a Dirichlet prior.

$$P(n \mid \alpha) = \frac{n!\,\Gamma(\alpha_0)}{\Gamma(n+\alpha_0)} \prod_j \frac{\Gamma(n_j+\alpha_j)}{n_j!\,\Gamma(\alpha_j)}$$

symbol		meaning	SI unit
`alpha`	in	prior concentrations (> 0)	—
`counts`	in	category counts	—
`probability`	out	P(counts | alpha)	—
`log_probability`	out	its natural log	—

Applies when: alpha_j > 0; counts whole numbers >= 0.

```
$ bayesian-statistics solve dirichlet_multinomial_evidence "alpha=[1, 1, 1]" "counts=[2, 1, 0]"
probability = 0.1
log_probability = -2.302585093
  by: Dirichlet-multinomial probability of a count vector (evidence)  (closed form with log-gamma
functions)
  assumes: alpha_j > 0; counts whole numbers >= 0
```

What the worked example means: This calculation determines the marginal likelihood of the specific category counts given the Dirichlet prior concentrations. It integrates over all possible probability vectors to find how likely this data structure is under the prior alone. The resulting evidence value, 0.1, indicates the fit between the prior assumptions and the observed integer counts, while its log scale, -2.30259, provides numerical stability for comparisons.

Gamma-exponential conjugate posterior of a rate — `gamma_exponential_posterior`

Update a Gamma(shape, rate) prior on the rate of exponential waiting times with observed times.

$$\lambda \mid t \sim \mathrm{Gamma}(\alpha + n,\ \beta + \textstyle\sum t_i)$$

symbol		meaning	SI unit
`shape`	in	prior shape (> 0)	—
`rate`	in	prior rate (> 0)	—
`times`	in	observed waiting times (> 0)	—
`post_shape`	out	shape + n	—

symbol		meaning	SI unit
`post_rate`	out	rate + sum of times	—
`posterior_mean`	out	posterior mean of the rate	—
`posterior_variance`	out	posterior variance	—
`posterior_sd`	out	posterior standard deviation	—

Applies when: shape, rate > 0; times > 0.

```
$ bayesian-statistics solve gamma_exponential_posterior "times=[0.5, 1.2, 0.8]" shape=2 rate=1
post_shape = 5
post_rate = 3.5
posterior_mean = 1.428571429
posterior_variance = 0.4081632653
posterior_sd = 0.638876565
  by: Gamma-exponential conjugate posterior of a rate  (conjugate update of the gamma prior with
the exponential likelihood)
  shape = 2   []
  rate = 1   []
  assumes: shape, rate > 0; times > 0
```

What the worked example means: This calculation updates the uncertainty about the event rate by combining prior beliefs with new data. Starting with a Gamma prior defined by shape 2 and rate 1, the sum of observed waiting times is incorporated. The resulting posterior distribution has a shape of 5 and a rate of 3.5. Consequently, the expected value for the rate parameter is estimated at 1.42857, reflecting a blend of initial assumptions and empirical evidence.

Gamma-Poisson conjugate posterior of a rate — `gamma_poisson_posterior`

Update a Gamma(shape, rate) prior on a Poisson rate with observed counts: posterior parameters, mean, variance and standard deviation.

$$\lambda \mid x \sim \mathrm{Gamma}(\alpha + \textstyle\sum x_i,\ \beta + n)$$

symbol		meaning	SI unit
`shape`	in	prior shape alpha (> 0)	—
`rate`	in	prior rate beta (> 0)	—
`counts`	in	observed counts (whole numbers), one per unit interval	—
`post_shape`	out	alpha + sum of counts	—
`post_rate`	out	beta + number of intervals	—
`posterior_mean`	out	posterior mean of the rate	—
`posterior_variance`	out	posterior variance	—
`posterior_sd`	out	posterior standard deviation	—

Applies when: shape, rate > 0; counts whole numbers >= 0.

```
$ bayesian-statistics solve gamma_poisson_posterior "counts=[3, 5, 4]" shape=2 rate=1
post_shape = 14
post_rate = 4
posterior_mean = 3.5
posterior_variance = 0.875
posterior_sd = 0.9354143467
  by: Gamma-Poisson conjugate posterior of a rate  (conjugate update of the gamma prior with the
Poisson likelihood)
  shape = 2   []
  rate = 1   []
  assumes: shape, rate > 0; counts whole numbers >= 0
```

What the worked example means: This calculation updates a Gamma prior on a Poisson rate using observed counts. The new shape parameter, 14, combines the prior belief with the total evidence from the data. The updated rate, 4, reflects the number of intervals plus the prior rate. Consequently, the estimated average event rate is 3.5, with a standard deviation of 0.935414 indicating the uncertainty in this estimate.

Laplace's rule of succession — `rule_of_succession`

Probability that the next trial is a success after k successes in n trials, with a uniform prior on the success probability: (k + 1)/(n + 2).

$$P(\text{next success}) = \frac{k+1}{n+2}$$

symbol		meaning	SI unit
`successes`	in	number of successes k	—
`trials`	in	number of trials n	—
`p_next`	out	posterior predictive probability that the next trial succeeds	—
`mle`	out	observed proportion k/n (none when n = 0)	—

Applies when: 0 <= k <= n; uniform Beta(1, 1) prior.

```
$ bayesian-statistics solve rule_of_succession successes=7 trials=10
p_next = 0.6666666667
mle = 0.7
  by: Laplace's rule of succession  (posterior mean of Beta(k + 1, n - k + 1))
  assumes: 0 <= k <= n; uniform Beta(1, 1) prior
```

In the worked example, successes = 7, trials = 10 go through the equation above and give p_next = 0.6666666667, mle = 0.7. It applies only when 0 <= k <= n; uniform Beta(1, 1) prior: outside that it says what is wrong instead of returning a number.

MAP estimate of a Poisson rate (gamma prior) — `map_gamma_poisson`

Posterior mode of a Poisson rate with a Gamma(shape, rate) prior and observed counts, with the MLE and the posterior mean for comparison.

$$\hat{\lambda}_{MAP} = \frac{\alpha + \sum x_i - 1}{\beta + n}$$

symbol		meaning	SI unit
`shape`	in	prior shape (> 0)	—
`rate`	in	prior rate (> 0)	—
`counts`	in	observed counts	—
`map`	out	(shape + sum - 1)/(rate + n)	—
`mle`	out	sample mean of the counts	—
`posterior_mean`	out	(shape + sum)/(rate + n)	—

Applies when: shape + sum of counts > 1.

```
$ bayesian-statistics solve map_gamma_poisson "counts=[3, 5, 4]" shape=2 rate=1
map = 3.25
mle = 4
posterior_mean = 3.5
  by: MAP estimate of a Poisson rate (gamma prior)  (stationary point of the log posterior)
  shape = 2   []
  rate = 1   []
  assumes: shape + sum of counts > 1
```

What the worked example means: This calculation determines the most probable value for the Poisson rate parameter given observed counts and prior beliefs. The result, 3.25, represents the peak of the updated probability distribution. For context, the simple average of the data is 4, and the expected value of the distribution is 3.5. These values help compare how much the prior information influences the final estimate relative to the raw data.

MAP estimate of a proportion (beta-binomial) — `map_beta_binomial`

The posterior mode (maximum a posteriori estimate) of p after k successes in n trials with a Beta(alpha, beta) prior, with the maximum-likelihood estimate and the posterior mean for comparison.

$$\hat{p}_{MAP} = \frac{\alpha + k - 1}{\alpha + \beta + n - 2}$$

symbol		meaning	SI unit
`alpha`	in	prior Beta alpha (> 0)	—
`beta`	in	prior Beta beta (> 0)	—
`successes`	in	number of successes k	—
`trials`	in	number of trials n	—
`map`	out	posterior mode (alpha + k - 1)/(alpha + beta + n - 2)	—
`mle`	out	k/n	—
`posterior_mean`	out	(alpha + k)/(alpha + beta + n)	—

Applies when: alpha + k > 1 and beta + n - k > 1 (interior mode).

```
$ bayesian-statistics solve map_beta_binomial alpha=2 beta=2 successes=7 trials=10
map = 0.6666666667
mle = 0.7
posterior_mean = 0.6428571429
  by: MAP estimate of a proportion (beta-binomial)  (maximize the log posterior (closed form of its
stationary point))
  alpha = 2   []
  beta = 2   []
  assumes: alpha + k > 1 and beta + n - k > 1 (interior mode)
```

What the worked example means: This calculation determines the most probable value of a success rate given observed data and prior beliefs. Using the number of successes 7 out of 10 trials with prior parameters 2 and 2, the algorithm finds the peak of the posterior distribution. The resulting mode, 0.666667, represents the balance between the empirical frequency 0.7 and the prior expectation, while the posterior mean 0.642857 offers an alternative average-based estimate.

Normal mean with known variance: posterior and credible interval from a sample mean — `normal_normal_summary`

Posterior mean, standard deviation, credible interval and shrinkage factor of a normal mean from the sample mean of n observations with known sigma and a normal prior.

$$\mu_n = B\mu_0 + (1-B)\bar{x},\ B = \frac{\sigma^2/n}{\sigma^2/n + \tau_0^2}, \quad \mu_n \pm z_{(1+c)/2}\,\tau_n$$

symbol		meaning	SI unit
`prior_mean`	in	prior mean	—
`prior_sd`	in	prior standard deviation (> 0)	—
`sigma`	in	known standard deviation of one observation (> 0)	—
`sample_mean`	in	sample mean	—
`n`	in	sample size (>= 1)	—
`level`	in	credibility level as a fraction (0.95 for 95%; default 0.95) (optional)	—
`posterior_mean`	out	posterior mean	—
`posterior_sd`	out	posterior standard deviation	—
`lower`	out	lower credible limit	—
`upper`	out	upper credible limit	—
`shrinkage`	out	weight on the prior mean (sigma^2/n)/(sigma^2/n + tau0^2)	—

Applies when: prior sd > 0; sigma > 0; n >= 1; 0 < level < 1.

```
$ bayesian-statistics solve normal_normal_summary prior_mean=0 prior_sd=1 sigma=2 sample_mean=1 n=4
level=0.95
posterior_mean = 0.5
posterior_sd = 0.7071067812
lower = -0.8859038244
upper = 1.885903824
shrinkage = 0.5
  by: Normal mean with known variance: posterior and credible interval from a sample mean  (normal-
normal conjugate update, interval from normal quantiles)
  prior_mean = 0   []
  prior_sd = 1   []
  sigma = 2   []
  sample_mean = 1   []
  level = 0.95   []
  assumes: prior sd > 0; sigma > 0; n >= 1; 0 < level < 1
```

What the worked example means: This calculation updates the belief about the population mean using a normal prior and observed data summarized by the sample mean. The resulting posterior mean, 0.5, represents a weighted average of the prior mean and the sample mean, determined by the shrinkage factor 0.5. This factor indicates how much weight is given to the prior information versus the new data. The uncertainty in this estimate is quantified by the posterior standard deviation, 0.707107, which allows for the construction of a credible interval bounded by -0.885904 and 1.8859. This interval provides a range of plausible values for the true mean at the specified credibility level.

Normal mean with known variance: posterior from data — `normal_normal_posterior`

Posterior of a normal mean theta with a N(prior mean, prior sd^2) prior, given observations with known standard deviation sigma.

$$\frac{1}{\tau_n^2} = \frac{1}{\tau_0^2} + \frac{n}{\sigma^2}, \quad \mu_n = \tau_n^2 \left(\frac{\mu_0}{\tau_0^2} + \frac{n\bar{x}}{\sigma^2} \right)$$

symbol		meaning	SI unit
`prior_mean`	in	prior mean mu0	—
`prior_sd`	in	prior standard deviation tau0 (> 0)	—
`sigma`	in	known standard deviation of each observation (> 0)	—
`data`	in	the observations	—
`posterior_mean`	out	posterior mean	—
`posterior_sd`	out	posterior standard deviation	—
`posterior_variance`	out	posterior variance	—

Applies when: prior sd > 0; sigma > 0 known.

```
$ bayesian-statistics solve normal_normal_posterior prior_mean=10 prior_sd=2 sigma=3 "data=[12, 9,
14]"
posterior_mean = 10.95238095
posterior_sd = 1.309307341
posterior_variance = 1.714285714
  by: Normal mean with known variance: posterior from data  (precision-weighted average of the
prior mean and the sample mean)
  prior_mean = 10   []
  prior_sd = 2   []
  sigma = 3   []
  assumes: prior sd > 0; sigma > 0 known
```

What the worked example means: This calculation determines the updated belief about a normal mean given observed data. It combines the prior information, represented by a mean of 10 and a standard deviation of 2, with the evidence from the sample. The result is a posterior distribution characterized by a mean of 10.9524 and a standard deviation of 1.30931. This new estimate reflects a precision-weighted average, balancing the strength of the prior against the reliability of the data, which has a known observation standard deviation of 3.

Normal model with unknown mean and variance (normal-inverse-gamma) — `normal_inverse_gamma_posterior`

Conjugate normal-inverse-gamma update for normal data with unknown mean and variance: posterior parameters, posterior mean of the variance and the Student-t credible interval for the mean.

$$\beta_n = \beta_0 + \tfrac{1}{2}\sum (x_i - \bar{x})^2 + \frac{\kappa_0 n (\bar{x} - \mu_0)^2}{2(\kappa_0 + n)}, \quad \mu \mid x \sim t_{2\alpha_n}\left(\mu_n,\ \frac{\beta_n}{\alpha_n \kappa_n}\right)$$

symbol		meaning	SI unit
`mu0`	in	prior location of the mean	—
`kappa0`	in	prior pseudo-observations for the mean (> 0)	—
`alpha0`	in	prior inverse-gamma shape (> 0)	—
`beta0`	in	prior inverse-gamma scale (> 0)	—
`data`	in	the observations	—
`level`	in	credibility level as a fraction (0.95 for 95%; default 0.95) (optional)	—
`mu_n`	out	posterior location	—
`kappa_n`	out	kappa0 + n	—
`alpha_n`	out	alpha0 + n/2	—
`beta_n`	out	posterior scale	—
`sigma2_mean`	out	posterior mean of sigma^2 (alpha_n > 1)	—
`lower`	out	lower credible limit for the mean	—
`upper`	out	upper credible limit for the mean	—

Applies when: kappa0, alpha0, beta0 > 0; 0 < level < 1.

```
$ bayesian-statistics solve normal_inverse_gamma_posterior mu0=0 kappa0=1 alpha0=2 beta0=2 "data=
[1, -1]" level=0.95
mu_n = 0
kappa_n = 3
alpha_n = 3
beta_n = 3
sigma2_mean = 1.5
lower = -1.412725216
upper = 1.412725216
  by: Normal model with unknown mean and variance (normal-inverse-gamma)  (conjugate update;
marginal posterior of the mean is Student t with 2 alpha_n degrees of freedom)
  mu0 = 0   []
  kappa0 = 1   []
  alpha0 = 2   []
  beta0 = 2   []
  level = 0.95   []
  assumes: kappa0, alpha0, beta0 > 0; 0 < level < 1
```

What the worked example means: This update combines prior beliefs about the mean and variance with observed data to produce a joint posterior distribution. The posterior location of the mean is calculated as 0, reflecting a weighted average of the prior mean and sample mean. The expected value of the variance is 1.5, while the uncertainty in the mean is summarized by a Student t credible interval spanning from -1.41273 to 1.41273. These values indicate how the data has shifted the estimate relative to the initial assumptions.

Posterior predictive count (negative binomial) — `gamma_poisson_predictive`

Probability of exactly k events in the next unit interval when the Poisson rate has a Gamma(shape, rate) posterior; the cumulative probability and the predictive mean and variance.

$$P(X = k) = \frac{\Gamma(\alpha + k)}{\Gamma(\alpha)\, k!} \left(\frac{\beta}{\beta+1}\right)^{\alpha} \left(\frac{1}{\beta+1}\right)^{k}$$

symbol		meaning	SI unit
`shape`	in	posterior shape (> 0)	—
`rate`	in	posterior rate (> 0)	—
`events`	in	number of events k in the next interval	—
`probability`	out	P(exactly k events)	—
`prob_at_most`	out	P(at most k events)	—
`predictive_mean`	out	shape/rate	—
`predictive_variance`	out	shape/rate (1 + 1/rate)	—

Applies when: shape, rate > 0; k whole number >= 0.

```
$ bayesian-statistics solve gamma_poisson_predictive shape=1 rate=1 events=1
probability = 0.25
prob_at_most = 0.75
predictive_mean = 1
predictive_variance = 2
  by: Posterior predictive count (negative binomial)  (negative binomial distribution (the Poisson
averaged over the gamma posterior))
  shape = 1   []
  rate = 1   []
  assumes: shape, rate > 0; k whole number >= 0
```

What the worked example means: This calculation determines the likelihood of observing a specific number of future events by averaging the Poisson distribution over the Gamma posterior for the rate. Using a posterior shape of 1 and rate of 1, the model computes that the probability of exactly 1 events is 0.25. The cumulative probability of seeing at most 1 events is 0.75. Additionally, the expected count for the next interval is 1, and the variance in these counts is 2, reflecting uncertainty from both the Poisson process and the estimated rate.

Posterior predictive distribution of a new observation — `normal_posterior_predictive`

Mean, standard deviation and tail probabilities of the next observation in the normal model with known sigma, after a sample mean of n observations and a normal prior.

$$Y_{new} \mid \bar{x} \sim N\left(\mu_n,\ \tau_n^2 + \sigma^2\right)$$

symbol		meaning	SI unit
`prior_mean`	in	prior mean	—
`prior_sd`	in	prior standard deviation (> 0)	—
`sigma`	in	known standard deviation (> 0)	—
`sample_mean`	in	sample mean	—
`n`	in	sample size (>= 1)	—
`threshold`	in	value c for P(Y > c)	—
`predictive_mean`	out	posterior mean	—
`predictive_sd`	out	sqrt(posterior variance + sigma^2)	—
`prob_above`	out	P(Y_new > c)	—
`prob_below`	out	P(Y_new < c)	—

Applies when: prior sd > 0; sigma > 0; n >= 1.

```
$ bayesian-statistics solve normal_posterior_predictive prior_mean=0 prior_sd=1 sigma=2
sample_mean=1 n=4 threshold=0.5
predictive_mean = 0.5
predictive_sd = 2.121320344
prob_above = 0.5
prob_below = 0.5
  by: Posterior predictive distribution of a new observation  (predictive variance = posterior
variance + sampling variance)
  prior_mean = 0   []
  prior_sd = 1   []
  sigma = 2   []
  sample_mean = 1   []
  threshold = 0.5   []
  assumes: prior sd > 0; sigma > 0; n >= 1
```

What the worked example means: This calculation determines the distribution for a future observation by combining the uncertainty in the estimated mean with the inherent variability of the data. The posterior mean, which serves as the center of this predictive distribution, is computed as 0.5. This value reflects a weighted average of the prior belief at 0 and the sample evidence at 1. The spread of the prediction, given by the standard deviation 2.12132, accounts for both the precision of the mean estimate and the known noise level 2. Consequently, the probability that the next value falls below the threshold 0.5 is 0.5, while the chance it exceeds that threshold is 0.5. These results allow you to assess the likely range of future outcomes based on current knowledge.

Posterior predictive probability (beta-binomial) — `beta_binomial_predictive`

Probability of exactly j successes in m future trials when the success probability has a Beta(alpha, beta) posterior; the cumulative probability and the predictive mean.

$$P(J = j) = \binom{m}{j} \frac{B(\alpha + j,\ \beta + m - j)}{B(\alpha, \beta)}$$

symbol		meaning	SI unit
`alpha`	in	posterior alpha (> 0)	—
`beta`	in	posterior beta (> 0)	—
`future_trials`	in	number of future trials m (>= 1)	—
`future_successes`	in	number of future successes j	—
`probability`	out	P(exactly j successes in m trials)	—
`prob_at_most`	out	P(at most j)	—
`prob_more`	out	P(more than j)	—
`predictive_mean`	out	expected number of successes m alpha/(alpha+beta)	—

Applies when: alpha, beta > 0; 0 <= j <= m.

```
$ bayesian-statistics solve beta_binomial_predictive alpha=1 beta=1 future_successes=2
future_trials=4
probability = 0.2
prob_at_most = 0.6
prob_more = 0.4
predictive_mean = 2
  by: Posterior predictive probability (beta-binomial)  (beta-binomial distribution (the binomial
averaged over the beta posterior))
  alpha = 1   []
  beta = 1   []
  assumes: alpha, beta > 0; 0 <= j <= m
```

What the worked example means: This calculation determines the likelihood of observing a specific number of successes in upcoming experiments, given current uncertainty about the success rate. Using the beta-binomial distribution, it integrates over all possible success probabilities weighted by their posterior plausibility. The result shows that the chance of getting exactly 2 successes out of 4 trials is 0.2, while the cumulative probability of at most that many successes is 0.6. The expected number of successes under this model is 2.

Posterior probability that p exceeds a value — `beta_posterior_tail_probability`

P(p > c) and P(p < c) under a Beta(alpha, beta) posterior, optionally after k successes in n trials.

$$P(p > c \mid k) = 1 - I_c(\alpha + k,\ \beta + n - k)$$

symbol		meaning	SI unit
`alpha`	in	Beta alpha (> 0)	—
`beta`	in	Beta beta (> 0)	—
`threshold`	in	the value c, 0 < c < 1	—
`successes`	in	successes to add (default 0) (optional)	—
`trials`	in	trials to add (default 0) (optional)	—
`prob_above`	out	posterior probability P(p > c)	—
`prob_below`	out	posterior probability P(p < c)	—

Applies when: alpha, beta > 0; 0 < c < 1; 0 <= k <= n.

```
$ bayesian-statistics solve beta_posterior_tail_probability alpha=2 beta=1 threshold=0.5
prob_above = 0.75
prob_below = 0.25
  by: Posterior probability that p exceeds a value  (regularized incomplete beta function of the
(updated) posterior)
  alpha = 2   []
  beta = 1   []
  threshold = 0.5   []
  assumes: alpha, beta > 0; 0 < c < 1; 0 <= k <= n
```

What the worked example means: The posterior probability that p exceeds 0.5 is 0.75, while the probability it falls below is 0.25.

Probability that one Poisson rate exceeds another (gamma posteriors) — `probability_rate_greater_gamma`

P(lambda_1 > lambda_2) for independent Gamma(shape, rate) posteriors of two Poisson rates, and the ratio of their posterior means.

$$P(\lambda_1 > \lambda_2) = 1 - I_{b_1/(b_1+b_2)}(a_1, a_2)$$

symbol		meaning	SI unit
`shape1`	in	posterior shape of rate 1	—
`rate1`	in	posterior rate parameter of rate 1	—
`shape2`	in	posterior shape of rate 2	—
`rate2`	in	posterior rate parameter of rate 2	—
`prob_1_greater`	out	P(lambda_1 > lambda_2)	—
`prob_2_greater`	out	P(lambda_2 > lambda_1)	—
`rate_ratio_mean`	out	ratio of posterior means	—

Applies when: all parameters > 0.

```
$ bayesian-statistics solve probability_rate_greater_gamma shape1=5 rate1=1 shape2=5 rate2=1
prob_1_greater = 0.5
prob_2_greater = 0.5
rate_ratio_mean = 1
  by: Probability that one Poisson rate exceeds another (gamma posteriors)  (lambda_1 b_1 /
(lambda_1 b_1 + lambda_2 b_2) ~ Beta(shape1, shape2): one regularized incomplete beta)
  shape1 = 5   []
  rate1 = 1   []
  shape2 = 5   []
  rate2 = 1   []
  assumes: all parameters > 0
```

In the worked example, shape1 = 5, rate1 = 1, shape2 = 5, rate2 = 1 go through the equation above and give prob_1_greater = 0.5, prob_2_greater = 0.5, rate_ratio_mean = 1. It applies only when all parameters > 0: outside that it says what is wrong instead of returning a number.

Credible intervals (7)

Credible interval for a Poisson rate — `gamma_poisson_credible_interval`

Equal-tailed credible interval of a Poisson rate after observed counts with a Gamma(shape, rate) prior.

$$\left[\frac{G^{-1}_{\alpha'}\left(\frac{1-c}{2}\right)}{\beta'}, \frac{G^{-1}_{\alpha'}\left(\frac{1+c}{2}\right)}{\beta'}\right]$$

symbol		meaning	SI unit
`shape`	in	prior shape (> 0)	—
`rate`	in	prior rate (> 0)	—

symbol		meaning	SI unit
`counts`	in	observed counts	—
`level`	in	credibility level as a fraction (0.95 for 95%; default 0.95) (optional)	—
`post_shape`	out	posterior shape	—
`post_rate`	out	posterior rate	—
`lower`	out	lower limit	—
`upper`	out	upper limit	—

Applies when: shape, rate > 0; counts whole numbers >= 0; 0 < level < 1.

```
$ bayesian-statistics solve gamma_poisson_credible_interval "counts=[0]" shape=1 rate=1 level=0.9
post_shape = 1
post_rate = 2
lower = 0.02564664719
upper = 1.497866137
  by: Credible interval for a Poisson rate  (conjugate update, then gamma quantiles (inverse
regularized incomplete gamma))
  shape = 1   []
  rate = 1   []
  level = 0.9   []
  assumes: shape, rate > 0; counts whole numbers >= 0; 0 < level < 1
```

What the worked example means: This calculation determines an equal-tailed credible interval for the rate of events in a Poisson process. It updates a Gamma prior with observed counts to produce a posterior distribution characterized by a shape of 1 and a rate of 2. The resulting interval spans from 0.0256466 to 1.49787, indicating where the true rate is likely to lie given the data and prior beliefs.

Credible interval for a proportion (beta-binomial) — `beta_binomial_credible_interval`

Equal-tailed credible interval of a success probability after k successes in n trials with a Beta(alpha, beta) prior.

$$p \mid k \sim \mathrm{Beta}(\alpha + k, \beta + n - k), \quad [q_{(1-c)/2},\ q_{(1+c)/2}]$$

symbol		meaning	SI unit
`alpha`	in	prior Beta alpha (> 0)	—
`beta`	in	prior Beta beta (> 0)	—
`successes`	in	number of successes k	—
`trials`	in	number of trials n	—
`level`	in	credibility level as a fraction (0.95 for 95%; default 0.95) (optional)	—
`post_alpha`	out	posterior alpha	—
`post_beta`	out	posterior beta	—
`lower`	out	lower credible limit	—
`upper`	out	upper credible limit	—

Applies when: alpha, beta > 0; 0 <= k <= n; 0 < level < 1.

```
$ bayesian-statistics solve beta_binomial_credible_interval alpha=1 beta=1 successes=1 trials=1
level=0.9
post_alpha = 2
post_beta = 1
lower = 0.2236067978
upper = 0.9746794345
  by: Credible interval for a proportion (beta-binomial)  (conjugate update, then beta quantiles)
  alpha = 1   []
  beta = 1   []
  level = 0.9   []
  assumes: alpha, beta > 0; 0 <= k <= n; 0 < level < 1
```

In the worked example, alpha = 1, beta = 1, successes = 1, trials = 1, level = 0.9 go through the equation above and give post_alpha = 2, post_beta = 1, lower = 0.2236067978, upper = 0.9746794345. It applies only when alpha, beta > 0; 0 <= k <= n; 0 < level < 1: outside that it says what is wrong instead of returning a number.

Equal-tailed credible interval of a beta distribution — `beta_credible_interval`

The central (equal-tailed) credible interval and the median of a Beta(alpha, beta) posterior.

$$\left[F^{-1}_{\mathrm{Beta}(\alpha,\beta)}\left(\tfrac{1-c}{2}\right),\ F^{-1}_{\mathrm{Beta}(\alpha,\beta)}\left(\tfrac{1+c}{2}\right)\right]$$

symbol		meaning	SI unit
`alpha`	in	Beta alpha (> 0)	—
`beta`	in	Beta beta (> 0)	—
`level`	in	credibility level as a fraction (0.95 for 95%; default 0.95) (optional)	—
`lower`	out	lower limit (quantile (1 - level)/2)	—
`upper`	out	upper limit (quantile (1 + level)/2)	—
`median`	out	posterior median	—

Applies when: alpha, beta > 0; 0 < level < 1.

```
$ bayesian-statistics solve beta_credible_interval alpha=1 beta=1 level=0.9
lower = 0.05
upper = 0.95
median = 0.5
  by: Equal-tailed credible interval of a beta distribution  (beta quantiles by safeguarded Newton
inversion of the regularized incomplete beta function)
  alpha = 1   []
  beta = 1   []
  level = 0.9   []
  assumes: alpha, beta > 0; 0 < level < 1
```

What the worked example means: This calculation determines the equal-tailed credible interval and median for a Beta distribution posterior. Given shape parameters alpha of 1 and beta of 1, along with a credibility level of 0.9, the resulting interval is [0.05, 0.95]. The median, representing the most probable value, is 0.5. This range indicates where the true parameter value likely lies based on the observed data and prior beliefs.

Equal-tailed credible interval of a gamma distribution — `gamma_credible_interval`

The central credible interval and median of a Gamma(shape, rate) posterior.

$$\left[G^{-1}\left(\tfrac{1-c}{2}\right),\ G^{-1}\left(\tfrac{1+c}{2}\right)\right]$$

symbol		meaning	SI unit
`shape`	in	shape (> 0)	—
`rate`	in	rate (> 0)	—
`level`	in	credibility level as a fraction (0.95 for 95%; default 0.95) (optional)	—
`lower`	out	lower limit	—
`upper`	out	upper limit	—
`median`	out	median	—

Applies when: shape, rate > 0; 0 < level < 1.

```
$ bayesian-statistics solve gamma_credible_interval shape=1 rate=2 level=0.9
lower = 0.02564664719
upper = 1.497866137
median = 0.3465735903
  by: Equal-tailed credible interval of a gamma distribution  (gamma quantiles by inversion of the
regularized incomplete gamma function)
  shape = 1   []
  rate = 2   []
  level = 0.9   []
  assumes: shape, rate > 0; 0 < level < 1
```

What the worked example means: This calculation determines the equal-tailed credible interval and median for a Gamma distribution specified by shape 1 and rate 2 at credibility level 0.9. The resulting lower bound is 0.0256466 and the upper bound is 1.49787, enclosing the most plausible values for the parameter. The median, representing the point where half the probability mass lies below, is calculated as 0.346574. These values provide a quantitative measure of uncertainty around the estimated rate.

Highest-density interval of a beta posterior — `beta_hdi`

The shortest interval holding a stated posterior probability of a unimodal Beta(alpha, beta): equal density at both ends.

$$F(u) - F(l) = c, \quad f(l) = f(u)$$

symbol		meaning	SI unit
`alpha`	in	Beta alpha (> 1)	—
`beta`	in	Beta beta (> 1)	—
`level`	in	credibility level as a fraction (0.95 for 95%; default 0.95) (optional)	—
`lower`	out	lower limit	—
`upper`	out	upper limit	—

symbol		meaning	SI unit
`width`	out	interval width	—

Applies when: alpha > 1 and beta > 1 (unimodal, zero density at both ends); 0 < level < 1.

```
$ bayesian-statistics solve beta_hdi alpha=3 beta=9 level=0.95
lower = 0.04055516641
upper = 0.4837236593
width = 0.4431684929
  by: Highest-density interval of a beta posterior  (bisection on the lower limit l with u =
F^-1(F(l) + level) until f(l) = f(u))
  alpha = 3   []
  beta = 9   []
  level = 0.95   []
  assumes: alpha > 1 and beta > 1 (unimodal, zero density at both ends); 0 < level < 1
```

What the worked example means: This calculation determines the shortest interval containing the specified probability mass, defined by the level 0.95, for a Beta distribution with parameters alpha 3 and beta 9. The resulting lower limit is 0.0405552 and the upper limit is 0.483724, yielding an interval width of 0.443168. This range represents the most credible values where the density at both ends is equal.

Highest-density interval of a gamma posterior — `gamma_hdi`

The shortest interval holding a stated probability of a unimodal Gamma(shape, rate): equal density at both ends.

$$G(u) - G(l) = c, \quad g(l) = g(u)$$

symbol		meaning	SI unit
`shape`	in	shape (> 1)	—
`rate`	in	rate (> 0)	—
`level`	in	credibility level as a fraction (0.95 for 95%; default 0.95) (optional)	—
`lower`	out	lower limit	—
`upper`	out	upper limit	—
`width`	out	interval width	—

Applies when: shape > 1; rate > 0; 0 < level < 1.

```
$ bayesian-statistics solve gamma_hdi shape=5 rate=2 level=0.95
lower = 0.603480051
upper = 4.715108491
width = 4.11162844
  by: Highest-density interval of a gamma posterior  (bisection on the lower limit with u =
G^-1(G(l) + level) until the densities are equal)
  shape = 5   []
  rate = 2   []
  level = 0.95   []
  assumes: shape > 1; rate > 0; 0 < level < 1
```

What the worked example means: This calculation determines the shortest credible interval containing a specified probability mass for a unimodal gamma distribution. By ensuring that the density at the lower limit equals the density at the upper limit, it identifies the most plausible range of values. The resulting interval spans from 0.60348 to 4.71511, giving a total width of 4.11163 for the given shape and rate parameters.

Jeffreys interval for a binomial proportion — `jeffreys_interval`

The equal-tailed Bayesian interval for a proportion under the Jeffreys Beta(1/2, 1/2) prior (lower limit 0 when k = 0 and upper limit 1 when k = n).

$$\left[B^{-1}\left(\tfrac{1-c}{2};\, k+\tfrac{1}{2},\, n-k+\tfrac{1}{2}\right),\; B^{-1}\left(\tfrac{1+c}{2};\, k+\tfrac{1}{2},\, n-k+\tfrac{1}{2}\right)\right]$$

symbol		meaning	SI unit
`successes`	in	number of successes k	—
`trials`	in	number of trials n (>= 1)	—
`level`	in	credibility level as a fraction (0.95 for 95%; default 0.95) (optional)	—
`lower`	out	lower limit	—
`upper`	out	upper limit	—
`posterior_mean`	out	(k + 1/2)/(n + 1)	—

Applies when: 0 <= k <= n; n >= 1; 0 < level < 1.

```
$ bayesian-statistics solve jeffreys_interval successes=7 trials=50 level=0.95
lower = 0.06487170115
upper = 0.2553250282
posterior_mean = 0.1470588235
  by: Jeffreys interval for a binomial proportion  (quantiles of Beta(k + 1/2, n - k + 1/2))
  level = 0.95   []
  assumes: 0 <= k <= n; n >= 1; 0 < level < 1
```

What the worked example means: This calculation determines a Bayesian credible interval for a binomial proportion using the Jeffreys prior. Given 7 successes out of 50 trials at a credibility level of 0.95, the resulting interval ranges from 0.0648717 to 0.255325. The posterior mean of the proportion is calculated as 0.147059. This method provides an equal-tailed interval that naturally handles boundary cases, such as zero successes or all successes, without producing invalid limits.

Decision analysis (9)

Bayes action: minimum posterior expected loss — `bayes_action_minimum_expected_loss`

Posterior expected loss of each action from a loss matrix and (posterior) state probabilities; the Bayes action minimizes it.

$$\rho(a) = \sum_{s} P(s \mid x)\, L(a, s), \quad a^{*} = \arg\min_{a} \rho(a)$$

symbol		meaning	SI unit
`losses`	in	loss matrix: one row per action, one column per state	—
`probabilities`	in	posterior state probabilities (sum to 1)	—
`expected_losses`	out	expected loss of each action	—
`bayes_action`	out	action with least expected loss	—
`bayes_risk`	out	its expected loss	—

Applies when: probabilities sum to 1.

```
$ bayesian-statistics solve bayes_action_minimum_expected_loss "losses=[[0, 10], [5, 0]]"
"probabilities=[0.3, 0.7]"
expected_losses = [7.0,1.5]
bayes_action = 2
bayes_risk = 1.5
  by: Bayes action: minimum posterior expected loss  (expected loss per row, minimum)
  assumes: probabilities sum to 1
```

What the worked example means: This calculation determines the optimal decision by computing the expected loss for each possible action. The expected loss, such as [7, 1.5], is derived from the product of posterior state probabilities and the corresponding entries in the loss matrix. By comparing these values across all actions, the method identifies the specific action that yields the lowest risk, designated here as 2 with a minimum value of 1.5. This result provides a quantitative basis for choosing the safest course of action under uncertainty.

Best action by expected monetary value — `expected_value_decision`

Expected payoff of each action over uncertain states with known probabilities, and the action with the highest expected value.

$$EV(a) = \sum_{s} P(s)\,u(a,s), \quad a^{*} = \arg\max_{a} EV(a)$$

symbol		meaning	SI unit
`payoffs`	in	payoff matrix: one row per action, one column per state	—
`probabilities`	in	state probabilities (sum to 1)	—
`expected_values`	out	expected payoff of each action	—
`best_action`	out	number (1-based) of the best action (first if tied)	—
`best_expected_value`	out	its expected payoff	—

Applies when: probabilities sum to 1; one probability per column.

```
$ bayesian-statistics solve expected_value_decision "payoffs=[[100, -20], [40, 30]]"
"probabilities=[0.4, 0.6]"
expected_values = [28.0,34.0]
best_action = 2
best_expected_value = 34
  by: Best action by expected monetary value  (expected value of each row)
  assumes: probabilities sum to 1; one probability per column
```

What the worked example means: This calculation determines the optimal choice by computing the expected monetary value for each available action. It multiplies the payoff of every state by its corresponding probability and sums these products to find the average outcome per row. The result identifies that action number 2 yields the highest expected return, valued at 34, based on the provided payoff matrix and state probabilities.

Break-even (indifference) probability of two actions — `break_even_probability`

The probability of state 1 at which two actions have equal expected value, and which action is preferred above it.

$$p^* = \frac{u_{22} - u_{12}}{(u_{11} - u_{12}) - (u_{21} - u_{22})}$$

symbol		meaning	SI unit
`payoffs`	in	2 x 2 payoff matrix [[action 1 in state 1, in state 2], [action 2 ...]]	—
`break_even`	out	probability of state 1 at which the expected values are equal	—
`preferred_above`	out	action preferred when P(state 1) exceeds it	—
`expected_value_at_break_even`	out	the common expected value there	—

Applies when: a crossing strictly between 0 and 1.

```
$ bayesian-statistics solve break_even_probability "payoffs=[[100, -20], [40, 30]]"
break_even = 0.4545454545
preferred_above = 1
expected_value_at_break_even = 34.54545455
  by: Break-even (indifference) probability of two actions  (intersection of the two expected-value
lines)
  assumes: a crossing strictly between 0 and 1
```

What the worked example means: This calculation determines the specific probability of state one where the expected values of two actions intersect, denoted as 0.454545. Using the provided payoff matrix, the method identifies that action 1 yields a higher expected value whenever the probability of state one surpasses this critical threshold. The common expected value at this intersection point is 34.5455, representing the decision boundary between the two options.

Certainty equivalent with logarithmic utility — `certainty_equivalent_log_utility`

Expected log utility, certainty equivalent (the weighted geometric mean) and risk premium of a lottery over positive wealth levels.

$$CE = \exp\left(\sum_i p_i \ln x_i\right) = \prod_i x_i^{p_i}$$

symbol		meaning	SI unit
`outcomes`	in	positive wealth levels	—
`probabilities`	in	their probabilities (sum to 1)	—

symbol		meaning	SI unit
`expected_log_utility`	out	E[ln x]	—
`certainty_equivalent`	out	exp(E[ln x])	—
`risk_premium`	out	E[x] - CE	—
`expected_value`	out	E[x]	—

Applies when: outcomes > 0; probabilities sum to 1.

```
$ bayesian-statistics solve certainty_equivalent_log_utility "outcomes=[100, 400]" "probabilities=
[0.5, 0.5]"
expected_log_utility = 5.298317367
certainty_equivalent = 200
risk_premium = 50
expected_value = 250
  by: Certainty equivalent with logarithmic utility  (weighted geometric mean)
  assumes: outcomes > 0; probabilities sum to 1
```

What the worked example means: This calculation determines the certainty equivalent for a decision maker with logarithmic utility, which represents the guaranteed wealth level providing the same satisfaction as the risky lottery. The result, 200, is derived from the expected log utility, 5.29832. It serves as a weighted geometric mean of the positive outcomes, [100, 400], weighted by their probabilities, [0.5, 0.5]. The difference between the expected value, 250, and this certainty equivalent quantifies the risk premium, 50, indicating how much value is lost due to uncertainty.

Decision threshold probability (treat / do not treat) — `treatment_threshold`

The probability above which acting (treating) has lower expected loss than not acting, from the loss of a false positive and of a false negative; the decision for a given probability.

$$p^* = \frac{L_{FP}}{L_{FP} + L_{FN}}$$

symbol		meaning	SI unit
`loss_fp`	in	loss of acting when the condition is absent (false positive)	—
`loss_fn`	in	loss of not acting when the condition is present (false negative)	—
`probability`	in	probability of the condition (optional) (optional)	—
`threshold`	out	loss_fp / (loss_fp + loss_fn)	—
`decision`	out	treat / do not treat (if p given)	—

Applies when: losses > 0.

```
$ bayesian-statistics solve treatment_threshold loss_fp=1 loss_fn=9 probability=0.05
threshold = 0.1
decision = do not treat
  by: Decision threshold probability (treat / do not treat)  (equal expected losses)
  loss_fp = 1   []
  loss_fn = 9   []
  probability = 0.05   []
  assumes: losses > 0
```

What the worked example means: This calculation determines the critical probability level, 0.1, at which the expected loss of treating equals the expected loss of not treating. It is derived from the false positive loss, 1, and the false negative loss, 9. If a specific condition probability, 0.05, is provided, the system compares it to the threshold to output a recommendation of do not treat. A higher false positive cost raises the bar for action, while a higher false negative cost lowers it.

Decision under uncertainty: maximin, maximax, Laplace, minimax regret, Hurwicz — `decision_criteria_under_uncertainty`

The best action and its criterion value when state probabilities are unknown, by a stated criterion.

$$\textbf{maximin: } \max_{a} \min_{s} u, \quad \text{regret } r(a,s) = \max_{a'} u(a',s) - u(a,s)$$

symbol		meaning	SI unit
`payoffs`	in	payoff matrix: one row per action, one column per state	—
`criterion`	in	maximin, maximax, laplace, minimax regret or hurwicz	—
`optimism`	in	Hurwicz optimism coefficient alpha (Hurwicz only) (optional)	—
`scores`	out	criterion score of each action (maximum regret for minimax regret)	—
`best_action`	out	number of the chosen action (first if tied)	—
`criterion_value`	out	criterion value of the chosen action	—

Applies when: 0 <= alpha <= 1 for Hurwicz.

```
$ bayesian-statistics solve decision_criteria_under_uncertainty "payoffs=[[100, -20], [40, 30],
[60, 10]]" criterion=maximin
scores = [-20.0,30.0,10.0]
best_action = 2
criterion_value = 30
  by: Decision under uncertainty: maximin, maximax, Laplace, minimax regret, Hurwicz  (row minima /
maxima / means / Hurwicz mixtures, or maximum regret per row)
  criterion = maximin
  assumes: 0 <= alpha <= 1 for Hurwicz
```

What the worked example means: This calculation identifies the optimal decision when state probabilities are unknown, using the specified maximin criterion. It evaluates each action based on its payoff matrix row to compute scores such as minimums, maximums, means, or regrets. The resulting best action is 2, which achieves a score of 30. This process helps select a strategy that aligns with a specific risk attitude, ranging from pessimistic to optimistic, without requiring probability estimates for the possible states of nature.

Expected utility and certainty equivalent (exponential utility) — `certainty_equivalent_exponential_utility`

Expected utility, certainty equivalent and risk premium of a lottery for a risk-averse decision maker with exponential utility u(x) = 1 - exp(-x/R).

$$CE = -R \ln \sum_i p_i e^{-x_i/R}$$

symbol		meaning	SI unit
`outcomes`	in	monetary outcomes	—
`probabilities`	in	their probabilities (sum to 1)	—
`risk_tolerance`	in	risk tolerance R (> 0)	—
`expected_utility`	out	E[1 - exp(-x/R)]	—
`certainty_equivalent`	out	-R ln E[exp(-x/R)]	—
`risk_premium`	out	expected value - certainty equivalent	—
`expected_value`	out	expected value	—

Applies when: probabilities sum to 1; R > 0.

```
$ bayesian-statistics solve certainty_equivalent_exponential_utility "outcomes=[1000, 0]"
"probabilities=[0.5, 0.5]" risk_tolerance=500
expected_utility = 0.4323323584
certainty_equivalent = 283.1095848
risk_premium = 216.8904152
expected_value = 500
  by: Expected utility and certainty equivalent (exponential utility)  (log-sum-exp evaluation of
E[exp(-x/R)])
  risk_tolerance = 500   []
  assumes: probabilities sum to 1; R > 0
```

What the worked example means: This calculation determines the certainty equivalent for a risk-averse decision maker with exponential utility. Using an outcomes list and their corresponding probabilities, along with a risk tolerance parameter, it computes the expected utility as 0.432332. The certainty equivalent, representing the guaranteed amount that provides the same satisfaction as the risky prospect, is found to be 283.11. This value is derived from the formula involving the natural logarithm of the sum of probability-weighted exponentials. The difference between the expected value of 500 and this certainty equivalent constitutes the risk premium of 216.89, quantifying the cost of bearing uncertainty.

Expected value of perfect information (EVPI) — `expected_value_of_perfect_information`

The most a decision maker should pay to learn the true state before acting: expected value with perfect information minus the best expected value without it.

$$EVPI = \sum_s P(s) \max_a u(a,s) - \max_a \sum_s P(s)\, u(a,s)$$

symbol		meaning	SI unit
`payoffs`	in	payoff matrix: one row per action, one column per state	—
`probabilities`	in	state probabilities (sum to 1)	—
`evpi`	out	expected value of perfect information	—
`ev_with_perfect_information`	out	expected value of the best action in each state	—
`best_expected_value`	out	best expected value without information	—

Applies when: probabilities sum to 1.

```
$ bayesian-statistics solve expected_value_of_perfect_information "payoffs=[[100, -20], [40, 30]]"
"probabilities=[0.4, 0.6]"
evpi = 24
ev_with_perfect_information = 58
best_expected_value = 34
  by: Expected value of perfect information (EVPI)  (E[max over actions] - max over actions of E)
  assumes: probabilities sum to 1
```

What the worked example means: This calculation determines the maximum worth of eliminating uncertainty. It compares the expected payoff if you knew the exact state beforehand, which is 58, against the best expected payoff achievable with current knowledge, which is 34. The difference between these two figures is the EVPI, or 24. This value represents the highest price a rational decision maker should be willing to pay for perfect information about the state of nature.

Expected value of sample information (EVSI) — `expected_value_of_sample_information`

Value of an imperfect test before acting: for each possible signal, update the state probabilities by Bayes' theorem, choose the best action, and compare the expected value with that of acting now.

$$EVSI = \sum_{z} P(z) \max_{a} \sum_{s} P(s \mid z)\, u(a, s) - \max_{a} \sum_{s} P(s)\, u(a, s)$$

symbol		meaning	SI unit
`payoffs`	in	payoff matrix: one row per action, one column per state	—
`priors`	in	prior state probabilities (sum to 1)	—
`likelihoods`	in	likelihood matrix P(signal | state): one row per signal, one column per state	—
`evsi`	out	expected value of sample information	—
`ev_with_information`	out	expected value with the test	—
`ev_without_information`	out	best expected value without the test	—
`signal_probabilities`	out	probability of each signal	—
`best_action_per_signal`	out	best action after each signal	—

Applies when: priors sum to 1; each likelihood column sums to 1.

```
$ bayesian-statistics solve expected_value_of_sample_information "payoffs=[[50, -10], [0, 0]]"
"priors=[0.5, 0.5]" "likelihoods=[[0.9, 0.2], [0.1, 0.8]]"
evsi = 1.5
ev_with_information = 21.5
ev_without_information = 20
signal_probabilities = [0.55,0.45]
best_action_per_signal = [1,2]
  by: Expected value of sample information (EVSI)  (preposterior analysis: Bayes' theorem per
signal, best action per posterior, weighted by P(signal))
  assumes: priors sum to 1; each likelihood column sums to 1
```

What the worked example means: This calculation determines the worth of gathering data before making a decision. It compares the best possible outcome if you act now, which is 20, against the average best outcome if you first observe a signal and update your beliefs, which is 21.5. The difference between these two values is the expected value of sample information, 1.5. This figure tells you how much extra value you can expect to gain by using the test results to guide your choice among actions, based on the prior probabilities [0.5, 0.5] and the likelihoods of signals given states [0.9 0.2; 0.1 0.8].

Information and quadrature for Bayesian computation (3)

Gauss-Hermite rule for expectations under a normal distribution — `gauss_hermite_rule`

Nodes and weights of the n-point Gauss-Hermite rule for E[f(Z)], Z standard normal (probabilists' Hermite polynomials; weights sum to 1; exact for polynomials of degree up to 2n - 1). For N(mu, sigma^2) use mu + sigma x_i.

$$E[f(Z)] \approx \sum_{i=1}^{n} w_i f(x_i), \quad He_n(x_i) = 0, \quad w_i = \frac{n!}{n^2\, He_{n-1}(x_i)^2}$$

symbol		meaning	SI unit
`n`	in	number of nodes (1 to 40)	—
`nodes`	out	nodes x_i (ascending)	—
`weights`	out	weights w_i (sum to 1)	—

Applies when: 1 <= n <= 40.

```
$ bayesian-statistics solve gauss_hermite_rule n=3
nodes = [-1.73205080757,0.0,1.73205080757]
weights = [0.166666666667,0.666666666667,0.166666666667]
  by: Gauss-Hermite rule for expectations under a normal distribution  (Golub-Welsch: eigenvalues
of the Jacobi matrix (implicit QL), Newton-polished on He_n; weights n!/(n^2 He_{n-1}(x_i)^2))
  assumes: 1 <= n <= 40
```

In the worked example, n = 3 go through the equation above and give nodes = [-1.732050808, 0, 1.732050808], weights = [0.1666666667, 0.6666666667, 0.1666666667]. It applies only when 1 <= n <= 40: outside that it says what is wrong instead of returning a number.

Kullback-Leibler divergence between two beta distributions — `kl_divergence_beta`

KL(Beta(alpha1, beta1) || Beta(alpha2, beta2)): for example the information gained from a beta prior (distribution 2) to a beta posterior (distribution 1).

$$KL = \ln \frac{B(\alpha_2, \beta_2)}{B(\alpha_1, \beta_1)} + (\alpha_1 - \alpha_2)\psi(\alpha_1) + (\beta_1 - \beta_2)\psi(\beta_1) + (\alpha_2 - \alpha_1 + \beta_2 - \beta_1)\psi(\alpha_1 + \beta_1)$$

symbol		meaning	SI unit
`alpha1`	in	alpha of distribution 1	—
`beta1`	in	beta of distribution 1	—
`alpha2`	in	alpha of distribution 2	—
`beta2`	in	beta of distribution 2	—
`kl`	out	KL divergence in nats	—

Applies when: all parameters > 0.

```
$ bayesian-statistics solve kl_divergence_beta alpha1=2 beta1=1 alpha2=1 beta2=1
kl = 0.1931471806
  by: Kullback-Leibler divergence between two beta distributions  (closed form with log-beta and
digamma functions)
  alpha1 = 2   []
  beta1 = 1   []
  alpha2 = 1   []
  beta2 = 1   []
  assumes: all parameters > 0
```

What the worked example means: This calculation determines the information loss when approximating the first beta distribution with the second. By using the shape parameters alpha1, beta1, alpha2, and beta2, the closed-form method computes the asymmetric distance in nats. The resulting value of 0.193147 quantifies how much information is gained or lost between these two probability distributions representing proportions.

Kullback-Leibler divergence between two normal distributions — `kl_divergence_normal`

KL(N1 || N2) and KL(N2 || N1), e.g. the information gained from a normal prior to a normal posterior.

$$KL(N_1 \| N_2) = \ln \frac{\sigma_2}{\sigma_1} + \frac{\sigma_1^2 + (\mu_1 - \mu_2)^2}{2\sigma_2^2} - \frac{1}{2}$$

symbol		meaning	SI unit
`mean1`	in	mean of distribution 1	—
`sd1`	in	sd of distribution 1 (> 0)	—
`mean2`	in	mean of distribution 2	—
`sd2`	in	sd of distribution 2 (> 0)	—
`kl_1_to_2`	out	KL(N1 || N2) in nats	—
`kl_2_to_1`	out	KL(N2 || N1) in nats	—

Applies when: sd1, sd2 > 0.

```
$ bayesian-statistics solve kl_divergence_normal mean1=0 sd1=1 mean2=1 sd2=2
kl_1_to_2 = 0.4431471806
kl_2_to_1 = 1.306852819
  by: Kullback-Leibler divergence between two normal distributions  (closed form for two normals)
  mean1 = 0   []
  sd1 = 1   []
  mean2 = 1   []
  sd2 = 2   []
  assumes: sd1, sd2 > 0
```

What the worked example means: This calculation determines the Kullback-Leibler divergence between two normal distributions using their respective means and standard deviations. The result for the first direction, 0.443147, quantifies the information loss when approximating the first distribution with the second. Conversely, 1.30685 measures the asymmetry in the other direction. These values indicate how distinct the two probability models are, which is useful for comparing prior and posterior beliefs or assessing model fit differences.

Monte Carlo estimation (6)

Control-variate Monte Carlo estimate — `control_variate_estimate`

Variance-reduced estimate of E[Y] from paired draws (y_i, c_i) of the output and a control variate with known mean: optimal coefficient, estimate and the fraction of variance left (1 - r^2).

$$\hat{\mu}_{cv} = \bar{y} - \hat{b}\,(\bar{c} - \mu_c), \quad \hat{b} = \frac{\sum (y_i - \bar{y})(c_i - \bar{c})}{\sum (c_i - \bar{c})^2}$$

symbol		meaning	SI unit
`y`	in	simulated outputs y_i	—
`c`	in	control values c_i (same draws)	—
`control_mean`	in	known mean of the control	—
`estimate`	out	mean(y) - b (mean(c) - mu_c)	—
`coefficient`	out	b = cov(y, c)/var(c)	—
`plain_estimate`	out	mean(y)	—
`variance_reduction`	out	variance factor 1 - r^2	—

Applies when: outputs and control vary; n >= 3.

```
$ bayesian-statistics solve control_variate_estimate "y=[1, 2, 3]" "c=[1, 2, 3]" control_mean=2.5
estimate = 2.5
coefficient = 1
plain_estimate = 2
variance_reduction = 0
  by: Control-variate Monte Carlo estimate  (regression (optimal) control-variate coefficient)
  control_mean = 2.5   []
  assumes: outputs and control vary; n >= 3
```

What the worked example means: This calculation produces a variance-reduced Monte Carlo estimate of the expected output, denoted as 2.5. It adjusts the plain sample mean 2 using an optimal regression

coefficient 1 derived from paired draws of the output and a control variate with known mean 2.5. The resulting value represents a more precise approximation of the true expectation by accounting for the correlation between the simulated outputs and the control. The effectiveness of this adjustment is quantified by the variance reduction factor 0, indicating how much of the original sampling noise has been eliminated through this statistical technique.

Importance sampling estimate and effective sample size — `importance_sampling_estimate`

Self-normalized and plain importance-sampling estimates of E_p[f] from draws of a proposal q, with the effective sample size of the weights p/q.

$$\hat{\mu} = \frac{\sum_i w_i f(x_i)}{\sum_i w_i},\ w_i = \frac{p(x_i)}{q(x_i)}, \quad ESS = \frac{(\sum w_i)^2}{\sum w_i^2}$$

symbol		meaning	SI unit
`values`	in	f(x_i) at the draws	—
`target_density`	in	target density p(x_i) (or unnormalized)	—
`proposal_density`	in	proposal density q(x_i) > 0	—
`estimate`	out	self-normalized estimate sum w f / sum w	—
`unnormalized_estimate`	out	mean of w f (needs normalized densities)	—
`effective_sample_size`	out	(sum w)^2 / sum w^2	—

Applies when: q > 0; p >= 0, not all zero.

```
$ bayesian-statistics solve importance_sampling_estimate "values=[0, 1]" "target_density=[0.5,
0.5]" "proposal_density=[0.25, 0.75]"
estimate = 0.25
unnormalized_estimate = 0.3333333333
effective_sample_size = 1.6
  by: Importance sampling estimate and effective sample size  (weights w = p/q)
  assumes: q > 0; p >= 0, not all zero
```

What the worked example means: This calculation determines the importance sampling estimate of the target expectation using samples from a proposal distribution. The weights are computed as the ratio of the target density to the proposal density, resulting in a normalized estimate of 0.25. Additionally, the effective sample size, which measures the statistical efficiency of the weighted samples, is calculated as 1.6.

Monte Carlo estimate of an integral from uniform samples — `monte_carlo_integration`

Estimate of the integral of f over [a, b] from f evaluated at n points drawn uniformly on [a, b], with its Monte Carlo standard error.

$$\hat{I} = \frac{b-a}{n}\sum_i f(x_i), \quad SE = (b-a)\frac{s_f}{\sqrt{n}}$$

symbol		meaning	SI unit
`values`	in	f(x_i) at uniform sample points	—

symbol		meaning	SI unit
`lower`	in	lower limit a	—
`upper`	in	upper limit b (> a)	—
`estimate`	out	(b - a) times the mean of f	—
`standard_error`	out	(b - a) s / sqrt(n)	—
`n`	out	number of samples	—

Applies when: b > a; n >= 2.

```
$ bayesian-statistics solve monte_carlo_integration "values=[1, 3]" lower=0 upper=2
estimate = 4
standard_error = 2
n = 2
  by: Monte Carlo estimate of an integral from uniform samples  (sample mean of f scaled by the
interval length)
  lower = 0   []
  upper = 2   []
  assumes: b > a; n >= 2
```

What the worked example means: This calculation estimates the definite integral of a function over an interval by averaging the function values at uniformly sampled points and scaling that mean by the length of the interval. The result, 4, represents the area under the curve approximated by these random samples. The accompanying value, 2, quantifies the uncertainty of this approximation, indicating how much the estimate might vary if the sampling were repeated. This method relies on the law of large numbers, where increasing the number of samples generally improves accuracy.

Number of Monte Carlo draws for a target accuracy — `monte_carlo_sample_size`

How many independent draws make the half-width of the confidence interval of a Monte Carlo mean at most the stated value, given the standard deviation of one draw.

$$n = \left\lceil \left(\frac{z_{(1+c)/2}\,\sigma}{E} \right)^2 \right\rceil$$

symbol		meaning	SI unit
`sigma`	in	standard deviation of one draw (> 0)	—
`half_width`	in	target half-width (> 0)	—
`level`	in	credibility level as a fraction (0.95 for 95%; default 0.95) (optional)	—
`draws`	out	required number of draws (rounded up)	—
`z`	out	normal quantile z_{(1+c)/2}	—
`standard_error_target`	out	target standard error half_width/z	—

Applies when: sigma, half-width > 0; 0 < level < 1.

```
$ bayesian-statistics solve monte_carlo_sample_size sigma=10 half_width=0.5 level=0.95
draws = 1537
z = 1.959963985
standard_error_target = 0.2551067285
  by: Number of Monte Carlo draws for a target accuracy  (n = (z sigma / E)^2, rounded up)
  sigma = 10   []
  half_width = 0.5   []
  level = 0.95   []
  assumes: sigma, half-width > 0; 0 < level < 1
```

What the worked example means: This calculation determines the minimum number of independent samples needed for a Monte Carlo simulation to estimate a mean with a specified precision. By using the known or estimated standard deviation of a single draw, which is 10, and the target half-width of the confidence interval, set at 0.5, the method applies a normal quantile derived from the credibility level of 0.95. The resulting value of 1537 represents the count of draws required to ensure that the uncertainty in the estimated mean does not exceed the desired threshold, assuming the sampling distribution behaves normally.

Point of the Halton low-discrepancy sequence — `halton_point`

Coordinates of the point with a given index in the (unscrambled) Halton sequence of a given dimension: the radical inverses of the index in the first prime bases (index 0 is the origin).

$$\phi_b(i) = \sum_k d_k\, b^{-k-1} \ \text{ for } i = \sum_k d_k b^k$$

symbol		meaning	SI unit
`index`	in	index of the point (0, 1, 2, ...)	—
`dimension`	in	dimension (1 to 20)	—
`point`	out	coordinates in [0, 1)	—
`bases`	out	prime bases used	—

Applies when: index >= 0; 1 <= dimension <= 20.

```
$ bayesian-statistics solve halton_point index=5 dimension=3
point = [0.625,0.777777777778,0.04]
bases = [2.0,3.0,5.0]
  by: Point of the Halton low-discrepancy sequence  (radical inverse: reverse the base-b digits of
the index behind the radix point)
  assumes: index >= 0; 1 <= dimension <= 20
```

What the worked example means: This calculation determines the coordinates of a specific point in a Halton low-discrepancy sequence. By applying the radical inverse method to the integer index 5 for each of the first 3 prime bases, the algorithm generates a set of values [0.625, 0.777778, 0.04] within the unit interval. These coordinates are derived from reversing the base-b digits of the index behind the radix point, ensuring uniform coverage of the space defined by the selected bases [2, 3, 5].

Stratified sampling estimate and standard error — `stratified_sampling_estimate`

Estimate of a mean from strata with known weights, the sample mean, standard deviation and size of each stratum, and its standard error.

$$\hat{\mu} = \sum_h W_h \bar{y}_h, \quad SE = \sqrt{\sum_h W_h^2 s_h^2 / n_h}$$

symbol		meaning	SI unit
`weights`	in	stratum weights (sum to 1)	—
`means`	in	stratum sample means	—
`sds`	in	stratum sample standard deviations	—
`sizes`	in	stratum sample sizes	—
`estimate`	out	sum W_h mean_h	—
`standard_error`	out	sqrt(sum W_h^2 s_h^2 / n_h)	—

Applies when: weights sum to 1; sizes >= 1.

```
$ bayesian-statistics solve stratified_sampling_estimate "weights=[0.6, 0.4]" "means=[10, 20]"
"sds=[2, 5]" "sizes=[30, 20]"
estimate = 14
standard_error = 0.4979959839
  by: Stratified sampling estimate and standard error  (weighted stratum means)
  assumes: weights sum to 1; sizes >= 1
```

What the worked example means: This calculation determines the overall population mean by combining the sample means from each stratum, weighted by their respective proportions. The resulting estimate of 14 reflects the central tendency across all groups, while the standard error of 0.497996 quantifies the precision of this estimate based on the variability and size within each stratum.

Odds and likelihood ratios (4)

Likelihood ratios of a diagnostic test — `likelihood_ratios`

Positive and negative likelihood ratios and the diagnostic odds ratio from sensitivity and specificity.

$$LR^+ = \frac{\mathrm{Se}}{1 - \mathrm{Sp}}, \quad LR^- = \frac{1 - \mathrm{Se}}{\mathrm{Sp}}$$

symbol		meaning	SI unit
`sensitivity`	in	sensitivity P(positive | condition)	—
`specificity`	in	specificity P(negative | no condition)	—
`lr_positive`	out	LR+ = sensitivity / (1 - specificity)	—
`lr_negative`	out	LR- = (1 - sensitivity) / specificity	—
`diagnostic_odds_ratio`	out	DOR = LR+ / LR-	—

Applies when: 0 < specificity < 1; sensitivity < 1.

```
$ bayesian-statistics solve likelihood_ratios sensitivity=0.9 specificity=0.8
lr_positive = 4.5
lr_negative = 0.125
diagnostic_odds_ratio = 36
  by: Likelihood ratios of a diagnostic test  (ratios of the probabilities of a result with and
without the condition)
  sensitivity = 0.9   []
  specificity = 0.8   []
  assumes: 0 < specificity < 1; sensitivity < 1
```

What the worked example means: This calculation determines how much a test result changes the probability of having a condition. Using a sensitivity of 0.9 and specificity of 0.8, it computes the positive likelihood ratio as 4.5 and the negative likelihood ratio as 0.125. The resulting diagnostic odds ratio is 36, which summarizes the overall discriminatory power of the test independent of disease prevalence.

Odds and log-odds of a probability — `probability_to_odds`

The odds p/(1-p), the odds against and the log-odds (logit) of a probability.

$$\text{odds} = \frac{p}{1-p}, \quad \text{logit}\, p = \ln \frac{p}{1-p}$$

symbol		meaning	SI unit
`probability`	in	probability p, strictly between 0 and 1	—
`odds`	out	odds in favour p/(1-p)	—
`odds_against`	out	odds against (1-p)/p	—
`log_odds`	out	natural log-odds ln(p/(1-p))	—

Applies when: 0 < p < 1.

```
$ bayesian-statistics solve probability_to_odds probability=0.2
odds = 0.25
odds_against = 4
log_odds = -1.386294361
  by: Odds and log-odds of a probability  (definition of odds)
  probability = 0.2   []
  assumes: 0 < p < 1
```

What the worked example means: This calculation transforms a single probability value into three related measures. Given an input probability of 0.2, it computes the odds in favour as 0.25, which represents the ratio of success to failure. It also determines the odds against as 4, reflecting the inverse relationship, and calculates the natural logarithm of the odds, known as the logit or log-odds, resulting in -1.38629. These conversions allow bounded probabilities to be expressed on scales suitable for multiplicative updates.

Post-test probability from a likelihood ratio — `posterior_from_likelihood_ratio`

Bayes' theorem in odds form: post-test odds = pre-test odds x likelihood ratio, and the post-test probability.

$$\frac{P(H \mid E)}{1 - P(H \mid E)} = \frac{P(H)}{1 - P(H)} \times LR$$

symbol		meaning	SI unit
`prior`	in	pre-test (prior) probability, 0 < p < 1	—
`likelihood_ratio`	in	likelihood ratio (>= 0)	—
`posterior`	out	post-test (posterior) probability	—
`prior_odds`	out	pre-test odds	—
`posterior_odds`	out	post-test odds	—

Applies when: 0 < prior < 1; likelihood ratio >= 0.

```
$ bayesian-statistics solve posterior_from_likelihood_ratio prior=0.2 likelihood_ratio=6
posterior = 0.6
prior_odds = 0.25
posterior_odds = 1.5
  by: Post-test probability from a likelihood ratio  (odds form of Bayes' theorem)
  prior = 0.2   []
  likelihood_ratio = 6   []
  assumes: 0 < prior < 1; likelihood ratio >= 0
```

What the worked example means: This calculation updates the initial belief using Bayes theorem in odds form. Starting with a pre-test probability of 0.2, this converts to pre-test odds of 0.25. These odds are multiplied by the likelihood ratio of 6 to yield post-test odds of 1.5. Finally, these odds are converted back into a probability, resulting in a post-test probability of 0.6. This method simplifies updating beliefs when new evidence arrives.

Probability from odds — `odds_to_probability`

The probability o/(1+o) that corresponds to odds o in favour.

$$p = \frac{o}{1 + o}$$

symbol		meaning	SI unit
`odds`	in	odds in favour (>= 0)	—
`probability`	out	probability o/(1+o)	—
`complement`	out	1 - probability	—

Applies when: odds >= 0.

```
$ bayesian-statistics solve odds_to_probability odds=3
probability = 0.75
complement = 0.25
  by: Probability from odds  (p = o/(1+o))
  odds = 3   []
  assumes: odds >= 0
```

What the worked example means: This calculation converts the given odds in favour, which are 3, into a corresponding probability using the formula where the probability equals the odds divided by one plus the odds. The resulting probability is 0.75, representing the likelihood of the event occurring, while the complement is 0.25, which represents the probability that the event does not occur.

Posterior approximation (1)

Laplace (normal) approximation of a beta posterior — `laplace_approximation_beta`

Approximate a Beta(alpha, beta) posterior by a normal distribution centred at its mode with variance -1/(second derivative of the log density); the approximate credible interval.

$$\hat{p} = \frac{\alpha - 1}{\alpha + \beta - 2}, \quad \sigma^2 = \left[\frac{\alpha - 1}{\hat{p}^2} + \frac{\beta - 1}{(1 - \hat{p})^2} \right]^{-1}$$

symbol		meaning	SI unit
`alpha`	in	Beta alpha (> 1)	—
`beta`	in	Beta beta (> 1)	—
`level`	in	credibility level as a fraction (0.95 for 95%; default 0.95) (optional)	—
`mode`	out	posterior mode	—
`laplace_sd`	out	Laplace standard deviation	—
`lower`	out	approximate lower limit	—
`upper`	out	approximate upper limit	—

Applies when: alpha > 1 and beta > 1.

```
$ bayesian-statistics solve laplace_approximation_beta alpha=3 beta=3 level=0.95
mode = 0.5
laplace_sd = 0.25
lower = 0.01000900386
upper = 0.9899909961
  by: Laplace (normal) approximation of a beta posterior  (second-order Taylor expansion of the log
posterior at its mode)
  alpha = 3   []
  beta = 3   []
  level = 0.95   []
  assumes: alpha > 1 and beta > 1
```

What the worked example means: This calculation approximates the Beta posterior distribution with a normal curve centered at its most likely value. Using the input parameters, it determines the mode as 0.5 and the spread as 0.25. These values define an approximate credible interval between 0.010009 and 0.989991, offering a simpler way to interpret uncertainty when the exact Beta shape is complex or when analytical integration is difficult.

Prior elicitation (1)

Beta prior from a mean and standard deviation — `beta_prior_from_mean_sd`

The Beta(alpha, beta) prior with a stated mean and standard deviation (method of moments) and its effective prior sample size alpha + beta.

$$\nu = \frac{m(1-m)}{s^2} - 1, \quad \alpha = m\nu, \quad \beta = (1-m)\nu$$

symbol		meaning	SI unit
`mean`	in	prior mean, 0 < m < 1	—
`sd`	in	prior standard deviation, sd^2 < m(1 - m)	—
`alpha`	out	alpha	—
`beta`	out	beta	—
`prior_sample_size`	out	alpha + beta	—

Applies when: 0 < mean < 1; 0 < sd^2 < mean (1 - mean).

```
$ bayesian-statistics solve beta_prior_from_mean_sd mean=0.3 sd=0.1
alpha = 6
beta = 14
prior_sample_size = 20
  by: Beta prior from a mean and standard deviation  (match the beta mean and variance)
  mean = 0.3   []
  sd = 0.1   []
  assumes: 0 < mean < 1; 0 < sd^2 < mean (1 - mean)
```

What the worked example means: This calculation determines the shape parameters for a Beta distribution that matches a specified prior mean of 0.3 and standard deviation of 0.1. Using the method of moments gives alpha as 6 and beta as 14. The sum of these two parameters represents the effective prior sample size, which is calculated here as 20, indicating the strength of this prior belief relative to observed data.

Sensitivity analysis (5)

Morris elementary-effects screening — `morris_elementary_effects`

Mean, mean absolute value (mu*) and standard deviation of the elementary effects from one-at-a-time trajectories: in each row, output j+1 is the model at the previous point with input j raised by delta.

$$EE_j = \frac{f(x + \Delta e_j) - f(x)}{\Delta}, \quad \mu_j^* = \frac{1}{r}\sum_t |EE_j^{(t)}|$$

symbol		meaning	SI unit
`outputs`	in	model outputs: one row per trajectory, k + 1 values (base point, then input 1, 2, ... raised in turn)	—
`delta`	in	step delta (> 0)	—
`mu`	out	mean elementary effect of each input	—
`mu_star`	out	mean absolute elementary effect	—
`sigma`	out	standard deviation of the elementary effects	—

Applies when: at least 2 trajectories; delta > 0.

```
$ bayesian-statistics solve morris_elementary_effects "outputs=[[10, 12], [11, 10]]" delta=1
mu = [0.5]
mu_star = [1.5]
sigma = [2.12132034356]
  by: Morris elementary-effects screening  (elementary effects EE_ij = (y_{i,j+1} - y_{ij})/delta)
  delta = 1   []
  assumes: at least 2 trajectories; delta > 0
```

What the worked example means: This calculation determines the influence of each input variable by analyzing trajectories across the parameter space. The mean absolute elementary effect, reported as [1.5], quantifies the overall magnitude of impact an input has on the model output. A higher value indicates that changing this input leads to larger shifts in results regardless of direction. The standard deviation, shown as [2.12132], reveals whether the effect depends on other variables or if it is consistent. These metrics help prioritize which factors need detailed analysis.

Sobol indices of a linear (additive) model — `sobol_indices_linear_model`

First-order Sobol indices of y = sum a_i x_i with independent inputs of given standard deviations (S_i = a_i^2 sigma_i^2 / Var y; they sum to 1).

$$S_i = \frac{a_i^2 \sigma_i^2}{\sum_j a_j^2 \sigma_j^2}$$

symbol		meaning	SI unit
`coefficients`	in	model coefficients a_i	—
`sds`	in	input standard deviations sigma_i (> 0)	—
`first_order`	out	first-order (= total) indices	—
`variance`	out	Var y	—

Applies when: independent inputs; sigma_i > 0.

```
$ bayesian-statistics solve sobol_indices_linear_model "coefficients=[2, 1, 0.5]" "sds=[1, 2, 4]"
first_order = [0.333333333333,0.333333333333,0.333333333333]
variance = 12
  by: Sobol indices of a linear (additive) model  (variance shares of the terms)
  assumes: independent inputs; sigma_i > 0
```

What the worked example means: This calculation determines the proportion of output variance attributable to each input variable in a linear additive model. Using the provided coefficients [2, 1, 0.5] and input standard deviations [1, 2, 4], the method computes the total variance 12 and then isolates the contribution of each term. The resulting first-order indices [0.333333, 0.333333, 0.333333] represent these shares, indicating how much of the uncertainty in the outcome is driven by specific inputs under the assumption of independence.

Sobol indices of a model with an interaction term — `sobol_indices_interaction_model`

First-order, interaction and total Sobol indices of y = a1 x1 + a2 x2 + a12 x1 x2 with independent normal inputs x1 ~ N(mean1, sd1^2), x2 ~ N(mean2, sd2^2).

$$V_1 = (a_1 + a_{12}\mu_2)^2\sigma_1^2,\ V_2 = (a_2 + a_{12}\mu_1)^2\sigma_2^2,\ V_{12} = a_{12}^2\sigma_1^2\sigma_2^2$$

symbol		meaning	SI unit
`a1`	in	coefficient of x1	—
`a2`	in	coefficient of x2	—
`a12`	in	interaction coefficient	—
`mean1`	in	mean of x1	—
`sd1`	in	sd of x1 (> 0)	—
`mean2`	in	mean of x2	—
`sd2`	in	sd of x2 (> 0)	—
`S1`	out	first-order index of x1	—
`S2`	out	first-order index of x2	—
`S12`	out	interaction index	—
`ST1`	out	total index of x1	—
`ST2`	out	total index of x2	—
`variance`	out	Var y	—

Applies when: independent normal inputs.

```
$ bayesian-statistics solve sobol_indices_interaction_model a1=0 a2=0 a12=1 mean1=0 sd1=1 mean2=0
sd2=1
S1 = 0
S2 = 0
S12 = 1
ST1 = 1
ST2 = 1
variance = 1
  by: Sobol indices of a model with an interaction term  (ANOVA decomposition of the bilinear
polynomial)
  a1 = 0   []
  a2 = 0   []
  a12 = 1   []
  mean1 = 0   []
  sd1 = 1   []
  mean2 = 0   []
  sd2 = 1   []
  assumes: independent normal inputs
```

In the worked example, a1 = 0, a2 = 0, a12 = 1, mean1 = 0, sd1 = 1, mean2 = 0, sd2 = 1 go through the equation above and give S1 = 0, S2 = 0, S12 = 1, ST1 = 1. It applies only when independent normal inputs: outside that it says what is wrong instead of returning a number.

Sobol indices of Sobol's G-function — `sobol_indices_g_function`

Exact first-order and total Sobol indices and the variance of the G-function prod_i (|4 x_i - 2| + a_i)/(1 + a_i), x_i uniform on [0, 1].

$$V_i = \frac{1}{3(1+a_i)^2},\ V = \prod_i (1+V_i) - 1,\ S_i = \frac{V_i}{V},\ S_{Ti} = \frac{V_i \prod_{j \neq i}(1+V_j)}{V}$$

symbol		meaning	SI unit
`a`	in	parameters a_i >= 0 (small a_i = important input)	—
`first_order`	out	first-order indices S_i	—
`total_order`	out	total indices ST_i	—
`variance`	out	variance of the function	—

Applies when: a_i >= 0; independent uniform inputs.

```
$ bayesian-statistics solve sobol_indices_g_function "a=[0, 0]"
first_order = [0.428571428571,0.428571428571]
total_order = [0.571428571429,0.571428571429]
variance = 0.7777777778
  by: Sobol indices of Sobol's G-function  (partial variances V_i = 1/(3(1 + a_i)^2) of the product
form)
  assumes: a_i >= 0; independent uniform inputs
```

What the worked example means: This calculation determines exact sensitivity indices for Sobol's G-function using specified parameters. The result shows a first-order index of [0.428571, 0.428571] and a total index of [0.571429, 0.571429], derived from a function variance of 0.777778. These values quantify how much each input contributes to the output uncertainty, with smaller parameter values typically indicating more influential inputs.

Sobol sensitivity indices of the Ishigami function — `sobol_indices_ishigami`

Exact first-order and total Sobol indices and the variance of the Ishigami test function f = sin x1 + a sin^2 x2 + b x3^4 sin x1 with x_i uniform on [-pi, pi].

$$V = \frac{a^2}{8} + \frac{b\pi^4}{5} + \frac{b^2\pi^8}{18} + \frac{1}{2},\ V_1 = \frac{1}{2}\left(1 + \frac{b\pi^4}{5}\right)^2,\ V_2 = \frac{a^2}{8},\ V_{13} = \frac{8b^2\pi^8}{225}$$

symbol		meaning	SI unit
`a`	in	parameter a (commonly 7)	—
`b`	in	parameter b (commonly 0.1)	—
`S1`	out	first-order index of x1	—
`S2`	out	first-order index of x2	—
`S3`	out	first-order index of x3	—
`ST1`	out	total index of x1	—
`ST2`	out	total index of x2	—
`ST3`	out	total index of x3	—
`variance`	out	variance of f	—

Applies when: inputs independent uniform on [-pi, pi].

```
$ bayesian-statistics solve sobol_indices_ishigami a=7 b=0.1
S1 = 0.3139051911
S2 = 0.4424111448
S3 = 0
ST1 = 0.5575888552
ST2 = 0.4424111448
ST3 = 0.2436836641
variance = 13.84458794
  by: Sobol sensitivity indices of the Ishigami function  (closed-form variance decomposition)
  a = 7   []
  b = 0.1   []
  assumes: inputs independent uniform on [-pi, pi]
```

What the worked example means: This calculation determines the exact global sensitivity indices for the Ishigami test function using closed-form variance decomposition. Given parameters a set to 7 and b set to 0.1, the method computes the total output variance as 13.8446. The resulting first-order indices, which measure the main effect of each input variable independently, are 0.313905 for the first variable, 0.442411 for the second, and 0 for the third. The total indices, which include all interactions involving that variable, are reported as 0.557589, 0.442411, and 0.243684 respectively.

Uncertainty propagation (14)

Covariance propagation through a linear map y = A x — `covariance_propagation_linear`

Covariance matrix of the outputs y = A x and their standard uncertainties, given the covariance matrix of x (first-order propagation with a sensitivity matrix).

$$\Sigma_y = A\,\Sigma_x\,A^{\mathsf{T}}$$

symbol		meaning	SI unit
`matrix`	in	sensitivity (Jacobian) matrix A: one row per output	—
`covariance`	in	covariance matrix of the inputs (symmetric, positive semi-definite)	—
`output_covariance`	out	A Sigma A^T	—
`uncertainties`	out	square roots of its diagonal	—

Applies when: covariance symmetric positive semi-definite; columns of A = inputs.

```
$ bayesian-statistics solve covariance_propagation_linear "matrix=[[1, 1], [1, -1]]" "covariance=
[[0.04, 0.01], [0.01, 0.09]]"
output_covariance = [[0.15,-0.05],[-0.05,0.11]]
uncertainties = [0.387298334621,0.331662479036]
  by: Covariance propagation through a linear map y = A x  (matrix product A Sigma A^T)
  assumes: covariance symmetric positive semi-definite; columns of A = inputs
```

What the worked example means: This calculation determines the uncertainty structure of linearly transformed variables. Given the input covariance matrix [0.04 0.01; 0.01 0.09] and the sensitivity matrix [1 1; 1 -1], the method computes the output covariance [[0.15, -0.05], [-0.05, 0.11]] using the product A Sigma A transpose. The resulting diagonal elements provide the standard uncertainties [0.387298, 0.331662] for each output variable, reflecting how input correlations and variances propagate through the transformation.

Effective degrees of freedom and expanded uncertainty (Welch-Satterthwaite) — `welch_satterthwaite_expanded_uncertainty`

Combined standard uncertainty, Welch-Satterthwaite effective degrees of freedom, the Student-t coverage factor and the expanded uncertainty at a stated coverage probability.

$$\nu_{eff} = \frac{u_c^4}{\sum_i u_i^4/\nu_i}, \quad U = t_{(1+p)/2,\,\nu_{eff}}\, u_c$$

symbol		meaning	SI unit
`uncertainties`	in	uncertainty components c_i u_i (> 0)	—
`dof`	in	degrees of freedom of each component (> 0)	—
`level`	in	credibility level as a fraction (0.95 for 95%; default 0.95) (optional)	—
`combined_uncertainty`	out	u_c	—
`effective_dof`	out	nu_eff	—
`coverage_factor`	out	k = t_{(1+p)/2, nu_eff}	—
`expanded_uncertainty`	out	U = k u_c	—

Applies when: components and degrees of freedom > 0; 0 < level < 1.

```
$ bayesian-statistics solve welch_satterthwaite_expanded_uncertainty "uncertainties=[1]" "dof=[10]"
level=0.95
combined_uncertainty = 1
effective_dof = 10
coverage_factor = 2.228138852
expanded_uncertainty = 2.228138852
  by: Effective degrees of freedom and expanded uncertainty (Welch-Satterthwaite)  (Welch-
Satterthwaite formula and the Student-t quantile at non-integer degrees of freedom)
  level = 0.95   []
  assumes: components and degrees of freedom > 0; 0 < level < 1
```

What the worked example means: This calculation determines the effective degrees of freedom and the expanded uncertainty for a measurement result. It combines individual uncertainty components to find the total standard uncertainty, which is 1. Using the Welch-Satterthwaite approximation, it computes the effective degrees of freedom as 10. Based on the requested credibility level, the coverage factor is determined to be 2.22814. Multiplying these values gives the final expanded uncertainty of 2.22814, representing the interval within which the true value is expected to lie with the stated probability.

Exact mean and standard deviation of a product of independent normals — `product_of_normals_exact_moments`

Exact moments of XY for independent normal X and Y, compared with first-order propagation (which misses the sd_x sd_y term).

$$\mathrm{Var}(XY) = \mu_x^2\sigma_y^2 + \mu_y^2\sigma_x^2 + \sigma_x^2\sigma_y^2$$

symbol		meaning	SI unit
`mean_x`	in	mean of X	—
`sd_x`	in	sd of X (> 0)	—
`mean_y`	in	mean of Y	—
`sd_y`	in	sd of Y (> 0)	—
`exact_mean`	out	mean_x mean_y	—
`exact_sd`	out	sqrt(mx^2 sy^2 + my^2 sx^2 + sx^2 sy^2)	—
`first_order_sd`	out	sqrt(mx^2 sy^2 + my^2 sx^2)	—

Applies when: independent normal inputs.

```
$ bayesian-statistics solve product_of_normals_exact_moments mean_x=10 sd_x=1 mean_y=5 sd_y=2
exact_mean = 50
exact_sd = 20.71231518
first_order_sd = 20.61552813
  by: Exact mean and standard deviation of a product of independent normals  (exact second moments
of a product)
  mean_x = 10   []
  sd_x = 1   []
  mean_y = 5   []
  sd_y = 2   []
  assumes: independent normal inputs
```

What the worked example means: This calculation determines the precise statistical moments for the product of two independent normal variables. Using the input means and standard deviations, it computes the exact expected value as 50 and the true spread as 20.7123. This result highlights that standard linear error propagation yields a lower bound of 20.6155 because it omits the interaction term between the variances of the two inputs.

Exact mean and variance of a polynomial of a normal variable — `polynomial_of_normal_moments`

Exact mean, variance and standard deviation of Y = c0 + c1 X + ... + c6 X^6 for X ~ N(mean, sd^2) (nonlinear uncertainty propagation without linearization).

$$E[Z^{2m}] = (2m-1)!!, \quad X = \mu + \sigma Z$$

symbol		meaning	SI unit
`coefficients`	in	coefficients c0, c1, ... (constant first, up to degree 6)	—
`mean`	in	mean of X	—
`sd`	in	standard deviation of X (> 0)	—
`mean`	out	E[Y]	—
`variance`	out	Var[Y]	—
`sd`	out	sd of Y	—

Applies when: sd > 0; degree <= 6.

```
$ bayesian-statistics solve polynomial_of_normal_moments "coefficients=[1, 2, 3]" mean=0 sd=1
mean = 4
variance = 22
sd = 4.69041576
  by: Exact mean and variance of a polynomial of a normal variable  (substitute X = mean + sd Z and
use the normal moments E[Z^k] = (k - 1)!! for even k)
  mean = 0   []
  sd = 1   []
  assumes: sd > 0; degree <= 6
```

What the worked example means: This calculation determines the precise statistical properties of a polynomial function applied to a normally distributed variable. By substituting the input with its mean and standard deviation components, it uses known normal moments to compute the expected value as 4, the variance as 22, and the standard deviation as 4.69042. This method provides an accurate result for nonlinear uncertainty propagation without relying on linear approximations or simulation.

Exact versus first-order propagation through exp(X) — `lognormal_propagation`

Mean and standard deviation of Y = exp(X) for normal X ~ N(mu, sigma^2), exactly (lognormal) and by first-order propagation, showing the bias of the linear approximation.

$$E[e^X] = e^{\mu+\sigma^2/2}, \quad \mathrm{SD}[e^X] = e^{\mu+\sigma^2/2}\sqrt{e^{\sigma^2}-1}$$

symbol		meaning	SI unit
`mu`	in	mean of X	—
`sigma`	in	standard deviation of X (> 0)	—
`exact_mean`	out	exp(mu + sigma^2/2)	—
`exact_sd`	out	sqrt(exp(sigma^2) - 1) exp(mu + sigma^2/2)	—
`first_order_mean`	out	exp(mu)	—
`first_order_sd`	out	exp(mu) sigma	—

Applies when: sigma > 0.

```
$ bayesian-statistics solve lognormal_propagation mu=0 sigma=1
exact_mean = 1.648721271
exact_sd = 2.161197416
first_order_mean = 1
first_order_sd = 1
  by: Exact versus first-order propagation through exp(X)  (lognormal moments; first-order Taylor
propagation)
  mu = 0   []
  sigma = 1   []
  assumes: sigma > 0
```

What the worked example means: This calculation determines the expected value and spread of an exponential transformation of a normal variable. Using the input mean 0 and standard deviation 1, it computes the exact lognormal mean as 1.64872 and the exact standard deviation as 2.1612. For comparison, the linear propagation method yields an approximate mean of 1 and an approximate standard deviation of 1.

The gap between these values illustrates the bias introduced when ignoring higher-order terms in the Taylor expansion.

Inverse-variance weighted mean of measurements — `inverse_variance_weighted_mean`

Combine measurements of one quantity with different standard uncertainties: weighted mean, its uncertainty, chi-square of consistency and Birge ratio (equal to the posterior under a flat prior).

$$\bar{x}_w = \frac{\sum x_i/u_i^2}{\sum 1/u_i^2}, \quad u(\bar{x}_w) = \left(\sum 1/u_i^2\right)^{-1/2}$$

symbol		meaning	SI unit
`values`	in	measured values	—
`uncertainties`	in	their standard uncertainties (> 0)	—
`weighted_mean`	out	sum(x_i/u_i^2)/sum(1/u_i^2)	—
`uncertainty`	out	1/sqrt(sum 1/u_i^2)	—
`chi_square`	out	sum((x_i - mean)^2/u_i^2)	—
`birge_ratio`	out	sqrt(chi^2/(n-1))	—

Applies when: u_i > 0; n >= 2.

```
$ bayesian-statistics solve inverse_variance_weighted_mean "values=[5.1, 4.9]" "uncertainties=[0.1,
0.1]"
weighted_mean = 5
uncertainty = 0.07071067812
chi_square = 2
birge_ratio = 1.414213562
  by: Inverse-variance weighted mean of measurements  (inverse-variance weighting)
  assumes: u_i > 0; n >= 2
```

What the worked example means: This calculation produces a single best estimate by combining multiple measurements, giving more weight to those with smaller uncertainties. The resulting weighted mean is 5, which represents the most probable value given the data. Its associated standard uncertainty is 0.0707107, indicating the precision of this combined result. To assess whether the individual measurements agree with each other, the chi-square statistic is computed as 2. If this value is large relative to the degrees of freedom, it suggests inconsistency among the inputs, which is quantified by the Birge ratio of 1.41421. A ratio near one implies the reported uncertainties are consistent with the observed scatter.

Type B standard uncertainty from a stated half-width — `type_b_standard_uncertainty`

Standard uncertainty of a quantity known only to lie within +/- a, for a rectangular, triangular or U-shaped (arcsine) distribution.

$$u = \frac{a}{\sqrt{3}}\ (\text{rect.}),\ \frac{a}{\sqrt{6}}\ (\text{tri.}),\ \frac{a}{\sqrt{2}}\ (\text{arcsine})$$

symbol		meaning	SI unit
`half_width`	in	half-width a of the interval (> 0)	—
`distribution`	in	rectangular, triangular or arcsine (U-shaped)	—
`standard_uncertainty`	out	a / divisor	—
`divisor`	out	sqrt(3), sqrt(6) or sqrt(2)	—

Applies when: a > 0.

```
$ bayesian-statistics solve type_b_standard_uncertainty half_width=0.5 distribution=rectangular
standard_uncertainty = 0.2886751346
divisor = 1.732050808
  by: Type B standard uncertainty from a stated half-width  (standard deviation of the assumed
distribution)
  half_width = 0.5   []
  distribution = rectangular
  assumes: a > 0
```

What the worked example means: This calculation determines the standard uncertainty by dividing the stated half-width 0.5 by a specific divisor based on the assumed distribution shape. The resulting standard uncertainty is 0.288675, representing the estimated spread of the quantity's true value around its central estimate.

Uncertainty of a function of one variable — `uncertainty_function`

Value and first-order standard uncertainty of f(x) for exp, ln, log10, sqrt, sin, cos, tan (x in radians) or 1/x.

$$u_f = |f'(x)|u_x$$

symbol		meaning	SI unit
`x`	in	value of x (radians for sin, cos, tan)	—
`ux`	in	standard uncertainty of x	—
`function`	in	exp, ln, log10, sqrt, sin, cos, tan or reciprocal	—
`value`	out	f(x)	—
`uncertainty`	out	|f'(x)| u(x)	—

Applies when: x inside the function's domain.

Not in `bayesian-statistics` 1.1.0 yet: use the Python package or the model's page in the browser.

What the worked example means: This calculation determines the result of applying a specific mathematical function to an input variable and estimates the associated uncertainty using first-order error propagation. The computed value, 0.916291, represents the transformed quantity, while the output uncertainty, 0.02, quantifies how much this result could vary given the input uncertainty, 0.05. This method assumes that the change in the function's output is linearly proportional to the change in its input near the current value.

Uncertainty of a linear combination of correlated inputs — `uncertainty_linear_correlated`

Combined standard uncertainty of y = sum c_i x_i when the inputs are correlated (correlation matrix), u_y^2 = sum_ij c_i c_j u_i u_j r_ij.

$$u_y^2 = \sum_i \sum_j c_i c_j u_i u_j r_{ij}$$

symbol		meaning	SI unit
`values`	in	input values	—
`uncertainties`	in	standard uncertainties	—
`correlation`	in	correlation matrix (ones on the diagonal, positive semi-definite)	—
`coefficients`	in	coefficients c_i (default 1) (optional)	—
`value`	out	y	—
`uncertainty`	out	u_y	—

Applies when: valid correlation matrix.

```
$ bayesian-statistics solve uncertainty_linear_correlated "values=[10, 20]" "uncertainties=[0.3,
0.4]" "correlation=[[1, 0.5], [0.5, 1]]" "coefficients=[1, 1]"
value = 30
uncertainty = 0.608276253
  by: Uncertainty of a linear combination of correlated inputs  (quadratic form c^T Sigma c (GUM
eq. 13))
  assumes: valid correlation matrix
```

What the worked example means: This calculation determines the combined standard uncertainty for a linear combination of correlated inputs. It uses the quadratic form of the sensitivity coefficients, individual standard uncertainties, and the correlation matrix to account for dependencies between variables. The resulting combined standard uncertainty is 0.608276, which reflects how the variability of the inputs propagates through the model to affect the final value 30.

Uncertainty of a power x^n — `uncertainty_power`

Value, standard uncertainty and relative uncertainty of y = x^n (first order).

$$\frac{u_y}{|y|} = |n| \, \frac{u_x}{|x|}$$

symbol		meaning	SI unit
`x`	in	value of x	—
`ux`	in	standard uncertainty of x (>= 0)	—
`exponent`	in	exponent n	—
`value`	out	x^n	—
`uncertainty`	out	|n x^(n-1)| u(x)	—
`relative_uncertainty`	out	|n| u(x)/|x|	—

Applies when: x > 0 for a non-integer exponent; x != 0 for n < 1.

```
$ bayesian-statistics solve uncertainty_power x=3 ux=0.1 exponent=2
value = 9
uncertainty = 0.6
relative_uncertainty = 0.06666666667
  by: Uncertainty of a power x^n  (first-order propagation)
  x = 3   []
  ux = 0.1   []
  exponent = 2   []
  assumes: x > 0 for a non-integer exponent; x != 0 for n < 1
```

What the worked example means: This calculation determines the result of raising a base value to a specified exponent, along with its associated standard and relative uncertainties using first-order propagation. Given the input value 3 with an uncertainty of 0.1 and an exponent of 2, the output value is 9. The absolute uncertainty on this result is 0.6, while the relative uncertainty is 0.0666667. This approach assumes that the input uncertainty is small enough that linear approximation holds, allowing practitioners to estimate how measurement error scales when variables are transformed by powers.

Uncertainty of a power-law product y = k x1^p1 x2^p2 … — `uncertainty_power_law`

Value and first-order uncertainty of a product of powers of independent positive inputs (relative uncertainties weighted by the exponents add in quadrature).

$$\frac{u_y}{|y|} = \sqrt{\sum_i \left(p_i \, \frac{u_i}{x_i} \right)^2}$$

symbol		meaning	SI unit
`coefficient`	in	exact constant k	—
`values`	in	positive input values x_i	—
`exponents`	in	exponents p_i	—
`uncertainties`	in	standard uncertainties u_i	—
`value`	out	y	—
`uncertainty`	out	u_y	—
`relative_uncertainty`	out	u_y/|y|	—

Applies when: x_i > 0; independent inputs.

```
$ bayesian-statistics solve uncertainty_power_law coefficient=0.5 "values=[2, 3]" "exponents=[1,
2]" "uncertainties=[0.02, 0.06]"
value = 9
uncertainty = 0.3710795063
relative_uncertainty = 0.04123105626
  by: Uncertainty of a power-law product y = k x1^p1 x2^p2 ...  (first-order propagation in
relative form)
  coefficient = 0.5   []
  assumes: x_i > 0; independent inputs
```

What the worked example means: This calculation determines the value and first-order uncertainty for a power-law product formed by multiplying powers of independent positive inputs. The result, 9, is derived from

the coefficient 0.5 and the input values [2, 3] raised to their respective exponents [1, 2]. The associated uncertainty 0.37108 reflects how the standard uncertainties [0.02, 0.06] propagate through the formula, weighted by the exponents. A relative uncertainty of 0.0412311 indicates the precision of this estimate under the assumption that all inputs are independent and strictly positive.

Uncertainty of a product and a quotient — `uncertainty_product_quotient`

Value and first-order standard uncertainty of a*b and a/b for independent a and b.

$$\frac{u_{ab}}{|ab|} = \frac{u_{a/b}}{|a/b|} = \sqrt{\left(\frac{u_a}{a}\right)^2 + \left(\frac{u_b}{b}\right)^2}$$

symbol		meaning	SI unit
`a`	in	value of a	—
`ua`	in	standard uncertainty of a (>= 0)	—
`b`	in	value of b (non-zero)	—
`ub`	in	standard uncertainty of b (>= 0)	—
`product`	out	a*b	—
`u_product`	out	u(a*b)	—
`quotient`	out	a/b	—
`u_quotient`	out	u(a/b)	—

Applies when: independent a and b; b != 0.

```
$ bayesian-statistics solve uncertainty_product_quotient a=12 ua=0.3 b=4 ub=0.2
product = 48
u_product = 2.683281573
quotient = 3
u_quotient = 0.1677050983
  by: Uncertainty of a product and a quotient  (first-order propagation (relative uncertainties add
in quadrature))
  a = 12   []
  ua = 0.3   []
  b = 4   []
  ub = 0.2   []
  assumes: independent a and b; b != 0
```

What the worked example means: This calculation determines the value and standard uncertainty for both the product and the quotient of two independent quantities, a and b. The resulting product is 48 with an associated uncertainty of 2.68328, while the quotient is 3 with an uncertainty of 0.167705. These results are derived using first-order error propagation, where relative uncertainties combine in quadrature to estimate the total spread around the central values.

Uncertainty of a product and quotient with correlation — `uncertainty_product_correlated`

First-order standard uncertainty of a*b and a/b when a and b are correlated with coefficient rho.

$$u_{ab}^2 = b^2 u_a^2 + a^2 u_b^2 + 2ab\,\rho\,u_a u_b$$

symbol		meaning	SI unit
`a`	in	value of a	—
`ua`	in	standard uncertainty of a	—
`b`	in	value of b (non-zero)	—
`ub`	in	standard uncertainty of b	—
`rho`	in	correlation coefficient of a and b (-1 to 1)	—
`product`	out	a*b	—
`u_product`	out	u(a*b)	—
`quotient`	out	a/b	—
`u_quotient`	out	u(a/b)	—

Applies when: -1 <= rho <= 1; b != 0.

```
$ bayesian-statistics solve uncertainty_product_correlated a=1 ua=0.1 b=1 ub=0.1 rho=-1
product = 1
u_product = 0
quotient = 1
u_quotient = 0.2
  by: Uncertainty of a product and quotient with correlation  (first-order propagation with the
covariance term)
  a = 1   []
  ua = 0.1   []
  b = 1   []
  ub = 0.1   []
  rho = -1   []
  assumes: -1 <= rho <= 1; b != 0
```

What the worked example means: This calculation determines the first-order standard uncertainty for both the product and the quotient of two correlated variables. Using the value of a as 1 with uncertainty 0.1, and b as 1 with uncertainty 0.1, the method applies propagation rules that include a covariance term based on the correlation coefficient -1. The resulting uncertainties, 0 for the product and 0.2 for the quotient, reflect how shared systematic errors influence the precision of these derived quantities.

Uncertainty of a sum or linear combination (independent inputs) — `uncertainty_sum`

Value and combined standard uncertainty of y = sum c_i x_i for independent inputs (default c_i = 1; use -1 for a difference).

$$u_y = \sqrt{\sum_i c_i^2 u_i^2}$$

symbol		meaning	SI unit
`values`	in	input values x_i	—
`uncertainties`	in	standard uncertainties u_i (>= 0)	—
`coefficients`	in	coefficients c_i (default all 1) (optional)	—
`value`	out	y = sum c_i x_i	—
`uncertainty`	out	u_y = sqrt(sum (c_i u_i)^2)	—

Applies when: independent inputs; u_i >= 0.

```
$ bayesian-statistics solve uncertainty_sum "values=[12.3, 4.5, 7.1]" "uncertainties=[0.2, 0.1,
0.3]"
value = 23.9
uncertainty = 0.3741657387
  by: Uncertainty of a sum or linear combination (independent inputs)  (root-sum-of-squares (GUM,
first order))
  assumes: independent inputs; u_i >= 0
```

What the worked example means: This calculation determines the total value and its associated standard uncertainty for a linear combination of independent inputs. The central value, shown as 23.9, is derived by summing the products of each input value and its coefficient. The uncertainty, displayed as 0.374166, represents the square root of the sum of the squared individual uncertainties scaled by their coefficients. This result quantifies the precision of the computed quantity based on the propagation of errors from the constituent parts.

Questions and reports: support@ai999x.com. AI999X — The Future Is Self-Improving.